

C++ 复习与 Linux 网络编程

完整学习手册 · 从语法回忆到手写 HTTP 服务器

配套工程：CppLearning (15 个可运行程序)

生成时间 2026-09-07 16:24

目录

C++ 复习与 Linux 网络编程 · 完整学习手册

这份文档怎么用

学习路线图

章节与程序对照表

环境准备

一、Visual Studio 2022 (Windows, 主环境)

二、WSL2 (学 epoll 必须)

三、编译选项说明

第 0 章 · C++ 语法回忆速通

0.1 编译链接模型 —— 先搞懂这个，报错才看得懂

0.2 指针 vs 引用 —— 最容易糊涂的地方

0.3 栈内存 vs 堆内存

0.4 类的六个特殊成员函数

0.5 继承与多态

0.6 模板

0.7 异常与 RAII

第 1 章 · C++11

1.1 auto —— 编译期类型推导

1.2 decltype

1.3 右值引用与移动语义 —— C++11 最重要的特性

值类别

为什么需要移动

实现移动

std::move 什么都没搬

1.4 完美转发

万能引用 (Forwarding Reference)

引用折叠

为什么需要 `std::forward`

1.5 统一初始化 {}

1.6 Lambda 表达式

Lambda 的本质

【坑】捕获的生命周期

1.7 智能指针

`unique_ptr` —— 独占所有权，默认选它

`shared_ptr` —— 引用计数共享

`weak_ptr` —— 打破循环引用

三条经验

1.8 变参模板

1.9 其它 C++11 特性速览

1.10 多线程

基本用法

互斥锁

条件变量

原子操作

异步

其它

第 2 章 · C++14

2.1 泛型 lambda

2.2 初始化捕获（移动捕获）

2.3 `decltype(auto)`

2.4 放宽的 `constexpr` —— 编译期查找表

2.5 为什么必须用 `make_unique`

第 3 章 · C++17

3.1 结构化绑定

3.2 `if / switch` 带初始化语句

3.3 CTAD 类模板参数推导

3.4 `if constexpr` —— 杀手级特性

3.5 折叠表达式

3.6 `std::optional`

3.7 `std::variant` —— 类型安全的 union

3.8 `std::string_view` —— 投入产出比最高的性能优化

3.9 `std::filesystem`

3.10 并行算法

3.11 其它 C++17 实用点

第 4 章 · C++20

4.1 Concepts —— 让模板报错变成人话

定义

四种用法 (效果完全相同)

为什么重要

4.2 Ranges —— 管道式算法

对比

算法直接吃容器

常用 views

4.3 协程

执行流程

4.4 三向比较 $<=>$

4.5 `std::span`

4.6 `std::format`

4.7 并发新工具

4.8 其它 C++20

4.9 Modules (了解即可)

第 5 章 · C++23

5.1 `std::expected` —— 带错误值的返回

5.2 `std::print` / `println`

5.3 Ranges 补全

5.4 deducing this —— 显式对象参数

5.5 其它

5.6 C++26 展望 (已定案)

第 6 章 · 标准库 (STL)

6.1 容器选型决策树

6.2 复杂度与性质速查

6.3 vector 的几个关键点

6.4 map 的坑与技巧

6.5 算法速查

6.6 字符串

6.7 时间 `<chrono>`

6.8 随机数 `<random>`

6.9 函数对象 `<functional>`

6.10 智能指针选型 (再强调一次)

第 7 章 · 设计模式 (现代 C++ 版)

7.1 RAII —— C++ 最重要的模式

7.2 Pimpl —— 隔断编译依赖

7.3 CRTP 与 类型擦除

CRTP (奇异递归模板模式) —— 静态多态

类型擦除 —— 值语义 + 多态 + 不需要继承

7.4 创建型

单例 (Meyers Singleton)

工厂 (注册式) —— 加新类型不用改工厂代码

建造者

7.5 结构型

7.6 策略 —— 三种实现方式, 怎么选

7.7 观察者 (信号槽)

7.8 命令 + 撤销/重做

7.9 状态机 (variant 版) —— 强烈推荐

7.10 访问者 (variant 版) 与「表达式问题」

7.11 责任链 (中间件管道)

7.12 模式选择速查

7.13 反模式提醒

第 8 章 · 网络基础: 什么是网络

8.1 从最基本的问题开始

8.2 分层模型 —— 为什么要分层

8.3 封装与解封装

8.4 IP 地址与端口

IP 地址

端口

【面试】四元组

8.5 字节序 (大端 / 小端) —— 网络编程第一个坑

四个转换函数 (必须背下来)

什么时候必须转换

8.6 地址结构 sockaddr —— 一个历史包袱

8.7 DNS 解析

8.8 TCP vs UDP

8.9 TCP 三次握手

8.10 TCP 四次挥手与 TIME_WAIT

8.11 TCP 状态机

8.12 socket API 全景

每个调用干了什么

8.13 常用 socket 选项

8.14 阻塞与非阻塞, 五种 IO 模型

五种 IO 模型 (《UNIX 网络编程》经典分类)

8.15 在 Windows 上学 Linux 网络编程

8.16 排障工具速查

常见现象与原因对照

第 9 章 · TCP 编程实战

9.1 服务端的固定套路

9.2 三种并发模型

模型一: iterative 单线程串行

模型二: thread 每连接一个线程

模型三: pool 线程池

彻底的解法

9.3 处理一条连接: 三个必须记住的细节

9.4 【重点】TCP 粘包与拆包

三种解决方案

9.5 客户端要点

9.6 实测数据 (回环地址)

第 10 章 · UDP 编程

10.1 和 TCP 的编程差异

10.2 UDP 有消息边界

10.3 UDP 必须自己处理的事

10.4 UDP 也可以 connect()

10.5 UDP 的独门能力: 广播与组播

第 11 章 · IO 多路复用

11.1 为什么需要它

11.2 三者对比

11.3 三种写法

select

poll

epoll (Linux)

11.4 【面试】水平触发 LT vs 边缘触发 ET

11.5 Reactor 模式

三种变体

11.6 事件驱动编程的实际难点

第 12 章 · 综合实战: 迷你 HTTP 服务器

12.1 HTTP 协议格式

12.2 解析器的关键: 可重入

12.3 架构: 中间件 + 路由

12.4 HTTP 关键概念

12.5 从这里往下走

附录 A · 四周复习计划

第 1 周: 语法回炉

第 2 周: 现代特性 + 标准库

第 3 周: 设计模式 + 网络基础

第 4 周: 网络编程实战

附录 B · 练习题 (由易到难)

B.1 语法与 STL

B.2 设计模式

B.3 网络编程

附录 C · 高频面试题清单

附录 D · 推荐资源

书

网站

视频 / 会议

开源项目（按学习难度排序）

附录 E · 工程速查

编译命令

CMake 最小模板

本工程的命令

C++ 复习与 Linux 网络编程 · 完整学习手册

面向「以前学过 C++，但很久没写、语法忘得差不多」的人。从语法回忆开始，一路走到能写出一个 HTTP 服务器。

配套代码：`CppLearning/` 工程，15 个可独立运行的程序，全部已在 Visual Studio 2022 上编译验证通过。

这份文档怎么用

每一章都是「先讲清楚为什么，再给能跑的代码，最后说用在哪」的结构。

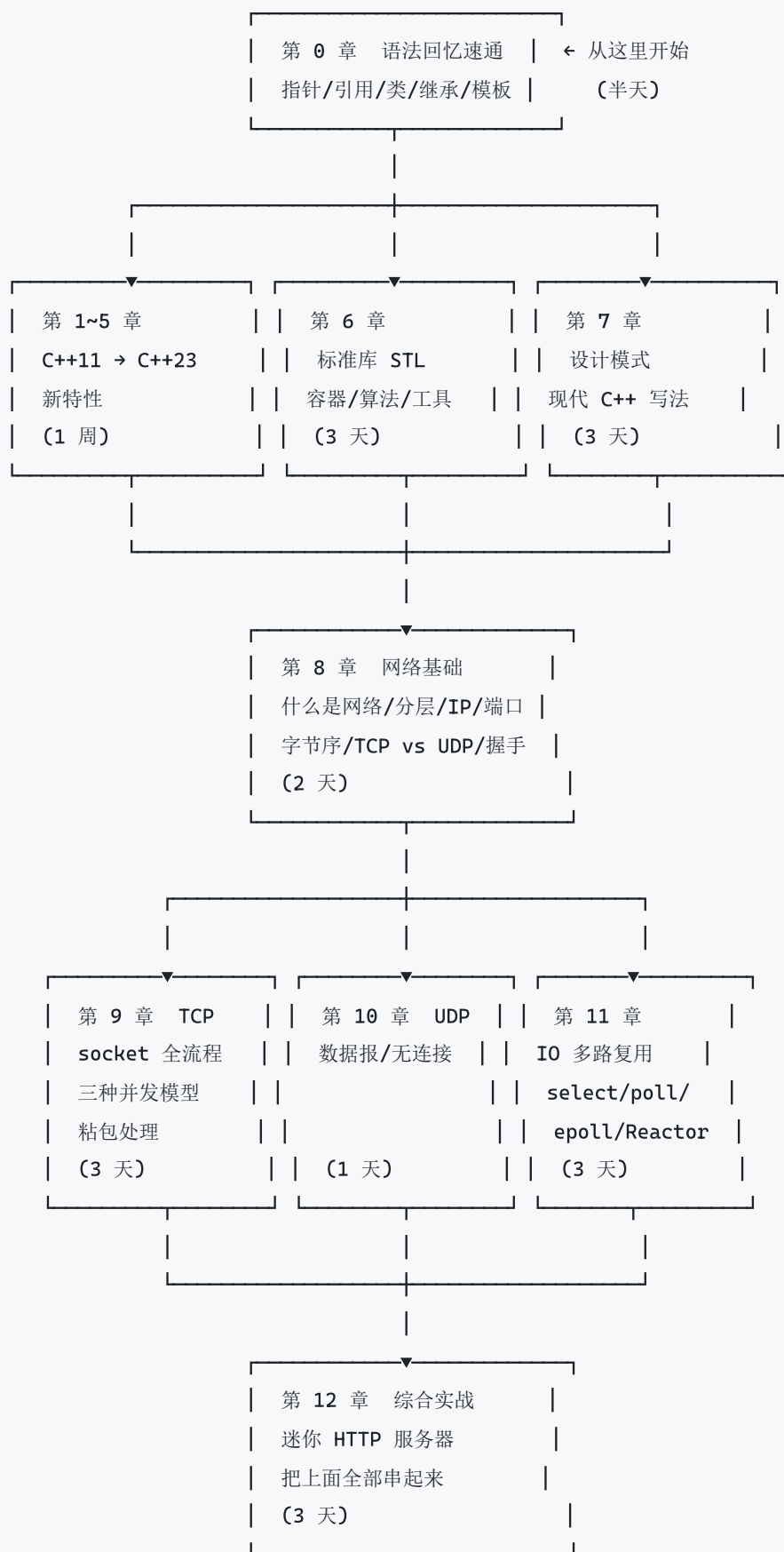
标记	含义
【用在哪】	这个特性在真实项目里的典型场景。看不懂原理没关系，先记住什么时候用
【坑】	实际写代码时最容易踩的地方
【面试】	高频面试考点
► <code>chXX_name</code>	对应的可运行程序，跑一遍比看十遍强

强烈建议的学习方式：

1. 先跑对应章节的程序，看输出
2. 再回来读文档解释
3. 然后打开源码，改几行，重新编译，看输出怎么变
4. 最后合上文档，自己默写一遍关键代码

只读不写，两周后忘光。

学习路线图



总计约 4 周（每天 2~3 小时）。赶时间的话：第 0 章 + 第 1/3/4 章 + 第 6 章 + 第 8/9/11 章，一周能过一遍主干。

章节与程序对照表

章	主题	可执行程序	说明
0	语法回忆速通	ch00_refresher	指针/引用/类/继承/模板/异常
1	C++11	ch01_cpp11	auto / 移动语义 / lambda / 智能指针 / 线程
2	C++14	ch02_cpp14	泛型 lambda / 移动捕获 / make_unique
3	C++17	ch03_cpp17	结构化绑定 / optional / variant / string_view
4	C++20	ch04_cpp20	concepts / ranges / 协程 / format / jthread
5	C++23	ch05_cpp23	expected / print / ranges 补全
6	标准库	ch06_stl	容器选型 / 算法 / 字符串 / 时间 / 随机数
7	设计模式	ch07_patterns	RAII / Pimpl / 工厂 / 观察者 / 状态机.....
8	网络基础	ch08_net_basics	分层 / 字节序 / 地址 / DNS / 握手挥手
9	TCP	ch09_tcp_server + ch09_tcp_client	回显服务，三种并发模型，粘包实测
10	UDP	ch10_udp_server + ch10_udp_client	数据报 / 消息边界 / 超时
11	IO 多路复用	ch11_multiplex	select / poll / epoll 聊天室
12	综合实战	ch12_http_server	迷你 HTTP 服务器

环境准备

一、Visual Studio 2022（Windows，主环境）

已确认本机安装：VS 2022 Community + MSVC 14.40，支持完整 C++20 与大部分 C++23。

方式 A：直接打开文件夹（推荐，最省事）

1. Visual Studio → 文件 → 打开 → 文件夹
2. 选择 CppLearning 目录
3. VS 会自动识别 CMakeLists.txt 并配置
4. 上方启动项下拉框里选任意一个 chXX_...exe，按 F5 运行

方式 B：生成 .sln 解决方案

```
scripts\build_vs.bat
```

跑完会得到：

- `build\CppLearning.sln` —— 双击用 VS 打开，左侧解决方案资源管理器里能看到 15 个项目
- `build\bin\*.exe` —— 所有可执行文件

在解决方案资源管理器里**右键某个项目** → **设为启动项目** → **F5**，就能单步调试那一章。

方式 C：命令行

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Debug
build\bin\ch01_cpp11.exe
```

二、WSL2 (学 epoll 必须)

`epoll` 是 Linux 独有的，Windows 上没有对应物（Windows 用 IOCP，模型完全不同）。第 11 章的 `epoll` 模式必须在 Linux 上跑。

```
# 管理员 PowerShell
wsl --install
```

重启后设置 Ubuntu 用户名密码，然后：

```
sudo apt update
sudo apt install -y build-essential gdb cmake git

# Windows 的 C 盘挂载在 /mnt/c
cd /mnt/c/Users/coder/Desktop/Claude_Code/CppLearning
cmake -B build-linux -DCMAKE_BUILD_TYPE=Debug
cmake --build build-linux -j$(nproc)

./build-linux/bin/ch11_multiplex 8890 epoll
```

Visual Studio 直接连 WSL 调试：安装「使用 C++ 的 Linux 开发」工作负载后，用「打开文件夹」方式打开工程，在配置下拉框里会多出 `WSL: Ubuntu` 目标，选它就能在 Linux 上编译 + 断点调试，代码不用动。

三、编译选项说明

本工程 `CMakeLists.txt` 里的关键设置，值得理解：

```
add_compile_options(  
    /utf-8          # 源文件和执行字符集都按 UTF-8 处理，中文注释和输出不乱码  
    /W4            # 高警告等级（建议平时就开着，能提前发现很多 bug）  
    /Zc:__cplusplus # 让 __cplusplus 宏报告真实版本（MSVC 默认一直报 199711L）  
    /permissive-    # 严格标准一致性，关掉 MSVC 的历史扩展  
    /EHsc          # 标准 C++ 异常模型  
)
```

GCC/Clang 侧对应的是 `-Wall -Wextra -Wpedantic`。

开发期强烈建议加上 sanitizer（GCC/Clang，MSVC 只支持 asan）：

```
g++ -std=c++20 -g -fsanitize=address,undefined main.cpp
```

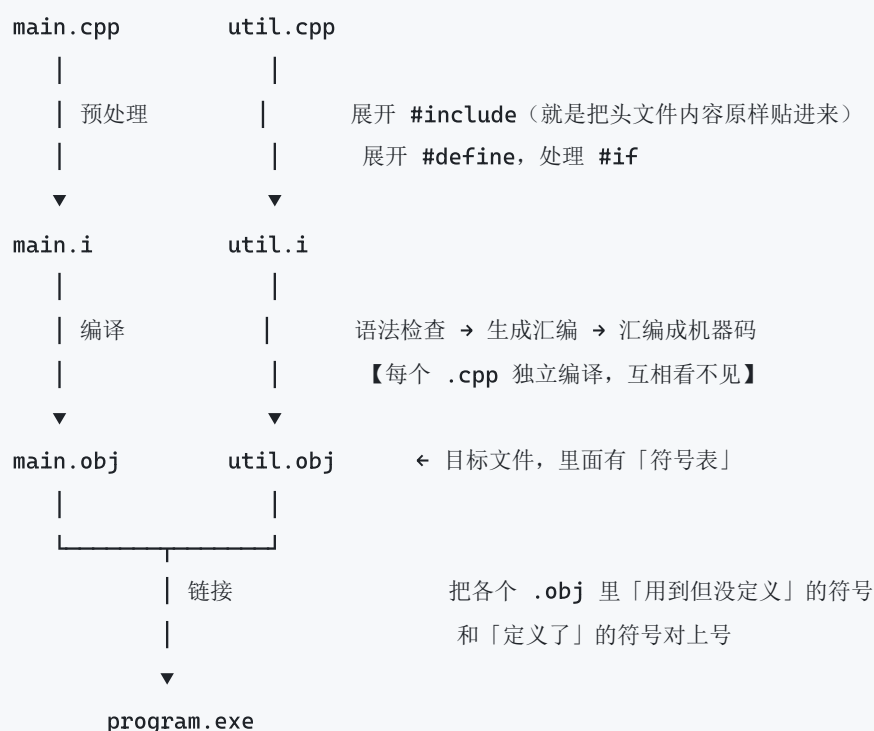
内存越界、use-after-free、未定义行为会在运行时直接报出精确位置，比对着调试器猜快一个数量级。

第 0 章 · C++ 语法回忆速通

► 对应程序：ch00_refresher

这一章把「以前会但忘了」的东西集中捡回来。只用 C++98/03 就有的语法，为后面新特性打底。

0.1 编译链接模型 —— 先搞懂这个，报错才看得懂



为什么要有 .h 和 .cpp 分开

- `.h` 放声明 ("有这么个东西, 长这样")
- `.cpp` 放定义 ("这个东西具体是什么")
- 每个 `.cpp` 独立编译, 只需要看到声明就能编译通过; 链接时才需要找到定义

两个最常见的链接错误

错误	含义	常见原因
LNK2019 无法解析的外部符号	声明了但找不到定义	忘了实现；忘了把 .cpp 加进工程；忘了链接第三方库；C 和 C++ 混用没加 <code>extern "C"</code>
LNK2005 重定义	同一个符号有多个定义	在头文件里定义了非 inline 的函数/变量，被多个 cpp 包含

头文件必须加保护

```
#pragma once           // 现代写法，所有主流编译器都支持

// 或者传统写法
#ifndef MY_HEADER_H
#define MY_HEADER_H
// ...
#endif
```

没有它，`a.h` 包含 `b.h`，`b.h` 又包含 `a.h`，就会无限递归或重复定义。

0.2 指针 vs 引用 —— 最容易糊涂的地方

```
int x = 10;
int* p = &x;    // 指针：变量 p 里【存的是 x 的地址】
int& r = x;     // 引用：r 就【是】x 的另一个名字
```

	指针 T*	引用 T&
可以为空	✓ <code>nullptr</code>	✗ 必须绑定到有效对象
必须初始化	✗ 可以先声明	✓ 声明时就必须绑定
可以改指向	✓ <code>p = &y;</code>	✗ <code>r = y;</code> 是给 x 赋值，不是重新绑定
有自己的地址	✓ <code>&p</code> 是指针的地址	✗ <code>&r</code> 就是 <code>&x</code>
需要解引用	✓ <code>*p</code>	✗ 直接用 <code>r</code>
可以有指针的指针	✓ <code>int**</code>	✗ 没有引用的引用

什么时候用哪个

```
// 参数: 能用引用就用引用, 语法干净且不用判空
void f(const std::string& s);      // 只读大对象 ← 最常用
void g(std::string& s);           // 要修改调用方的对象
void h(std::string* s);           // 「可以不传」时用指针 (传 nullptr 表示没有)

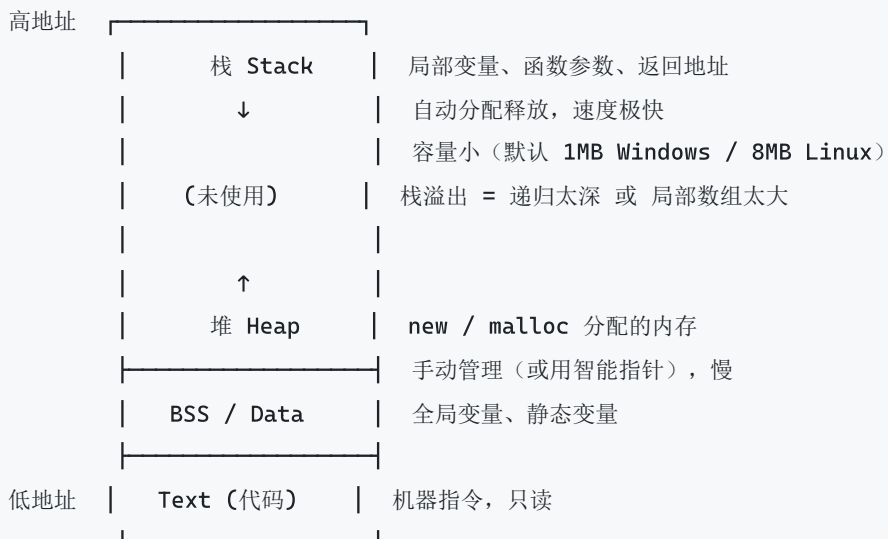
// 返回值: 绝对不要返回局部变量的引用或指针!
int& bad() { int x = 1; return x; } // 悬垂引用, 未定义行为
```

三种传参的实测对比 (ch00_refresher 里能看到输出)

```
void by_value(int x) { x = 100; }      // 改副本, 外面不变
void by_ref(int& x) { x = 200; }       // 改本体
void by_ptr(int* x) { if (x) *x = 300; } // 改本体, 但要判空

int v = 1;
by_value(v);    // v 还是 1
by_ref(v);      // v 变 200
by_ptr(&v);     // v 变 300
```

0.3 栈内存 vs 堆内存



```

void f() {
    int a = 1;           // 栈：函数返回自动销毁
    int arr[1000];       // 栈：4KB，还好
    // int huge[100000000]; // 栈：40MB，直接栈溢出崩溃

    int* p = new int(2); // 堆：必须 delete，否则泄漏
    delete p;

    int* arr2 = new int[100]; // 堆数组
    delete[] arr2;           // 注意是 delete[] 不是 delete!
}

```

现代 C++ 的态度：几乎不该出现裸的 `new / delete`。用 `std::vector`、`std::string`、`std::unique_ptr` —— 它们内部管堆，外部用起来像栈对象，离开作用域自动释放。

0.4 类的六个特殊成员函数

```

class Widget {
public:
    Widget();           // 1. 默认构造
    ~Widget();          // 2. 析构
    Widget(const Widget&); // 3. 拷贝构造
    Widget& operator=(const Widget&); // 4. 拷贝赋值
    Widget(Widget&&) noexcept; // 5. 移动构造 (C++11)
    Widget& operator=(Widget&&) noexcept; // 6. 移动赋值 (C++11)
};

```

编译器会自动生成它们，但有联动规则（很绕，记结论即可）：

- 你写了任何一个拷贝/移动/析构 → 移动操作**不会**自动生成
- 你写了移动操作 → 拷贝操作被**删除**

两条实用法则

零法则 (Rule of Zero)：优先让类不管理任何资源（成员全用 `vector` / `string` / 智能指针），这样六个函数一个都不用写，编译器生成的全都是对的。**这是首选。**

五法则 (Rule of Five)：如果确实要管裸资源（文件句柄、socket、C API 指针），那五个（析构 + 拷贝 ×2 + 移动 ×2）都要显式处理，缺一个都可能出 bug。

构造函数初始化列表 vs 函数体内赋值

```

class Foo {
    std::string name_;
    const int id_;
    Bar& ref_;
public:
    // ✓ 初始化列表：直接构造，一步到位
    Foo(std::string n, int i, Bar& b) : name_(std::move(n)), id_(i), ref_(b) {}

    // ✗ 函数体赋值：先默认构造再赋值，多一次操作
    // 而且 const 成员和引用成员【根本没法】在函数体里赋值
    Foo(std::string n) { name_ = n; /* id_ = ...; 编译错误 */ }
};

```

【坑】初始化顺序取决于成员的声明顺序，不是初始化列表的书写顺序。

```

class Bad {
    int b_;
    int a_;
public:
    Bad() : a_(1), b_(a_) {} // b_ 先初始化（声明在前），此时 a_ 还是垃圾值！
};

```

开 `/W4` 或 `-Wall` 编译器会警告，所以警告一定要开。

0.5 继承与多态

```

class Animal {
public:
    virtual ~Animal() = default; // 【必须】基类析构要 virtual
    virtual std::string speak() const = 0; // 纯虚 = 抽象接口
    virtual void describe() const { // 虚函数：有默认实现
        std::cout << speak();
    }
};

class Dog : public Animal {
public:
    std::string speak() const override { return "汪"; } // override 让编译器帮你查错
};

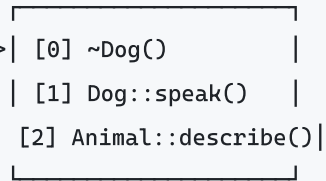
```

虚函数是怎么工作的

Dog 对象



Dog 的虚表 (vtable)



```
animal_ptr->speak();
```

```
实际执行: (*(animal_ptr->vptr[1]))(animal_ptr)
```

- **代价:** 对象多一个指针 (8 字节); 调用多一次间接跳转; 无法内联
- **收益:** 运行期多态 —— 一份代码处理所有派生类型

【坑 1】基类析构不是 virtual = 内存泄漏

```
Animal* p = new Dog();
delete p;           // 如果 ~Animal() 不是 virtual, 只会调 ~Animal(),
                    // Dog 特有的成员不会被析构 -> 泄漏
```

规则: **只要类可能被继承并通过基类指针 delete, 析构就必须 virtual。** 反之, 不打算被继承的类应该标 final。

【坑 2】对象切片 (Object Slicing)

```
void f(Animal a);           // 按值传基类!
Dog d;
f(d);                       // Dog 被"切"成 Animal, 派生部分丢失, 多态失效

void g(const Animal& a);    // ✅ 正确: 用引用或指针才有多态
```

【坑 3】**不要在构造函数/析构函数里调虚函数** 构造基类时派生类还没构造好, 虚表还指向基类 —— 调不到派生类的实现。

【面试】override 的价值

```
struct Base { virtual void f(int); };
struct Derived : Base {
    void f(long) override;    // 编译错误! 签名不匹配
    // 不写 override 的话, 编译器会当成【新增一个虚函数】,
    // 你调 base_ptr->f(1) 永远走不到这里, 能调试一整天
};
```

0.6 模板

```
// 函数模板
template <typename T>
T max_of(T a, T b) { return a > b ? a : b; }

// 类模板
template <typename T>
class Stack {
    std::vector<T> data_;
public:
    void push(const T& v) { data_.push_back(v); }
    const T& top() const { return data_.back(); }
};

// 全特化: 为特定类型换一套实现
template <>
class Stack<bool> { /* 位压缩版 */ };
```

模板是「编译期的代码生成器」

`Stack<int>` 和 `Stack<double>` 是**两个完全不同的类**，编译器为每个用到的类型 各生成一份代码（叫「实例化」）。这带来：

-  零运行期开销，全部可内联
 -  代码膨胀（每个类型一份）
 -  编译慢
 -  模板定义必须放头文件里（因为实例化时需要看到完整定义）
 -  C++20 之前报错信息极其恐怖（几百行）—— 这正是 concepts 要解决的问题
-

0.7 异常与 RAII

标准异常继承树

```
std::exception
├── std::logic_error      (程序逻辑错误, 本可避免)
│   ├── std::invalid_argument
│   ├── std::domain_error
│   ├── std::length_error
│   └── std::out_of_range
├── std::runtime_error    (运行期才能发现的错误)
│   ├── std::overflow_error
│   ├── std::underflow_error
│   ├── std::range_error
│   └── std::system_error (errno / OS 错误码)
├── std::bad_alloc        (new 失败)
└── std::bad_cast         (dynamic_cast 引用版失败)
```

```
try {
    may_throw();
} catch (const std::invalid_argument& e) { // 具体的放前面
    std::cout << e.what();
} catch (const std::exception& e) {      // 基类兜底放后面
    std::cout << e.what();
} catch (...) {                          // 什么都接 (很少用)
}
```

永远用 `const&` 接住异常 —— 按值接会切片, 按指针接是 C 的思路。

RAII 是 C++ 处理异常安全的唯一正解

```
void bad() {
    Lock* l = new Lock();
    may_throw(); // 抛异常 -> 下面那行不执行 -> 锁泄漏、死锁
    delete l;
}

void good() {
    std::lock_guard<std::mutex> l(mtx); // 栈对象
    may_throw(); // 抛异常 -> 栈展开 -> l 的析构一定被调用 -> 锁释放
}
```

核心机制叫「栈展开 (stack unwinding)」: 异常抛出后, 从抛出点到 catch 点之间 所有已构造的栈对象, 析构函数都会被依次调用。这是 C++ 相对 C 最大的优势之一。

【坑】析构函数绝对不能抛异常。 栈展开过程中如果析构又抛，直接 `std::terminate`。C++11 起析构函数默认就是 `noexcept`。

第 1 章 · C++11

► 对应程序：ch01_cpp11

C++11 是 C++ 历史上最大的一次升级，大到 Bjarne Stroustrup 说「感觉像一门新语言」。后面 C++14/17/20 很多东西都是在补 C++11 留下的洞，所以这一章值得花最多时间。

1.1 auto —— 编译期类型推导

```
auto i = 42;           // int
auto d = 3.14;         // double
auto s = std::string("hi"); // std::string
```

注意：auto 不是动态类型，类型在编译期就定死了，只是不用你手写。

推导规则（三条，会考）

```
const int ci = 10;

auto      a1 = ci;    // int          -- 顶层 const 被丢弃，因为是拷贝
const auto a2 = ci;    // const int    -- 自己加回来
auto&      a3 = ci;    // const int&    -- 引用会保留 const（不然就能绕过 const 了）
auto&&     a4 = ci;    // const int&    -- 万能引用，遇左值折叠成左值引用
```

【用在哪】

```
// 1. 迭代器 — auto 最初就是为这个发明的
for (auto it = m.begin(); it != m.end(); ++it) { }
// 对比: for (std::map<std::string, std::vector<int>>::const_iterator it = ...)

// 2. lambda — lambda 的类型是编译器生成的匿名类，你根本写不出来
auto f = [](int x) { return x * 2; };

// 3. 长得离谱的模板返回类型
auto result = some_factory<Foo, Bar, Baz>::create();

// 4. 范围 for
for (const auto& item : container) { }
```

什么时候不该用 `auto`：类型本身是重要信息的地方。比如 API 返回值、需要精确控制整数宽度的地方（`auto x = v.size();` 得到 `size_t`，和 `int` 比较会有符号警告）。

【坑】 `vector<bool>` 的代理对象

```
std::vector<bool> v{true, false};  
auto x = v[0];           // x 不是 bool! 是 std::vector<bool>::reference 代理对象  
bool y = v[0];           // 这样才对
```

1.2 `decltype`

推导表达式的类型，且不求值（只做类型分析，不真的执行）。

```
int x = 0;  
decltype(x) y = 1;      // int  
decltype((x)) z = x;    // int& ← 加一层括号就变引用!
```

规则：`decltype(变量名)` 得到变量声明的类型；`decltype(表达式)` 中，如果表达式是左值，得到 `T&`。`(x)` 是表达式不是变量名，所以是 `int&`。

【用在哪】泛型代码里描述「和某某一样的类型」：

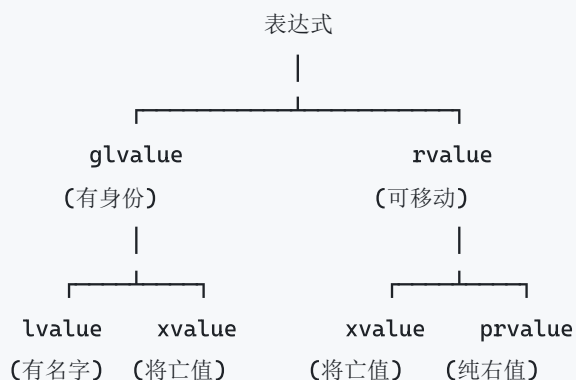
```
template <typename Container>  
auto get_first(Container& c) -> decltype(c[0]) {    // 返回类型跟着容器走  
    return c[0];  
}  
  
// 自定义删除器的类型  
auto deleter = [](FILE* f) { fclose(f); };  
std::unique_ptr<FILE, decltype(deleter)> fp(fopen("a", "r"), deleter);
```

尾置返回类型：`auto f(...) -> R`。必须这么写的场景是返回类型依赖参数：

```
template <typename T, typename U>  
auto add(T a, U b) -> decltype(a + b) { return a + b; }  
// 不能写成 decltype(a+b) add(T a, U b), 因为那时 a、b 还没进入作用域
```

1.3 右值引用与移动语义 —— C++11 最重要的特性

值类别



lvalue	: 有名字、能取地址	<code>x, *p, arr[0], ++x</code>
prvalue	: 临时的、马上就要死	<code>42, x+1, f()</code> 的返回值, <code>Foo{}</code>
xvalue	: 本来有名字但被判死刑	<code>std::move(x)</code> , 返回 T&& 的函数

能被移动的 = xvalue + prvalue = rvalue。

为什么需要移动

```
std::vector<std::string> make_big() {
    std::vector<std::string> v(10000, "长字符串...");
    return v;                      // C++03: 拷贝 10000 个字符串!
}
```

// C++11: 移动, 只搬 3 个指针

实现移动

```
class Buffer {
    size_t size_ = 0;
    char* data_ = nullptr;
public:
    // 拷贝构造：深拷贝，O(n)
    Buffer(const Buffer& o) : size_(o.size_), data_(new char[o.size_]) {
        std::memcpy(data_, o.data_, size_);
    }

    // 移动构造：偷指针，O(1)
    Buffer(Buffer&& o) noexcept
        : size_(o.size_), data_(o.data_) {
        o.data_ = nullptr;    // ★ 必须置空！否则析构两次 = 崩溃
        o.size_ = 0;
    }

    ~Buffer() { delete[] data_; }
};
```

★ noexcept 为什么关键

`std::vector` 扩容时要把老元素搬到新内存。它要保证「强异常安全」——搬到一半失败了，原来的 `vector` 必须完好无损。

- 移动构造**不是** `noexcept` → 搬到一半抛异常，老元素已经被掏空了，没法回滚 → **vector 宁可用拷贝**（拷贝失败老的还在）
- 移动构造是 `noexcept` → `vector` 放心用移动

所以：移动构造/移动赋值忘了写 `noexcept`，性能优化直接归零。

`std::move` 什么都没搬

```
template <class T>
constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(t);
}
```

它**只是一个类型转换**，把左值转成右值引用。真正干活的是随后被重载决议选中的移动构造/移动赋值函数。所以 `std::move` 更准确的名字应该叫 `rvalue_cast`。

【坑】被移动后的对象

标准规定：处于「有效但未指定（valid but unspecified）」状态。你可以**赋新值**或**析构**它，但**不能依赖它的值**。

```
std::string a = "hello";
std::string b = std::move(a);
// std::cout << a;    // 合法但内容不确定（多数实现是空串，但别依赖）
a = "world";          //  重新赋值是允许的
```

1.4 完美转发

万能引用（Forwarding Reference）

```
template <typename T>
void wrapper(T&& arg);    // 这里的 T&& 是【万能引用】，不是右值引用

void f(std::string&& s);   // 这里的 && 是【纯右值引用】
```

只有两种情况是万能引用：**T&&**（**T 是被推导的模板参数**）和 **auto&&**。

引用折叠

```
T&  &    -> T&
T&  &&   -> T&
T&& &    -> T&
T&& &&   -> T&&
```

口诀：只要有一个左值引用，结果就是左值引用。

```
int lv = 1;
wrapper(lv);    // T 推成 int& ,   T&& = int& && = int&
wrapper(42);    // T 推成 int  ,   T&& = int&&
```

为什么需要 `std::forward`

```
template <typename T>
void bad(T&& arg) {
    callee(arg);           // ❌ arg 是【具名变量】，它本身是左值！
}                          // 右值性丢失了

template <typename T>
void good(T&& arg) {
    callee(std::forward<T>(arg)); // ✅ T 是左值引用就转成左值，否则转成右值
}
```

`ch01_cpp11` 里能实测到这个差异：`bad_forward(42)` 打印「收到左值引用」（错误），`good_forward(42)` 打印「收到右值引用」（正确）。

【用在哪】所有「转发型」函数：

```
// make_unique 的原理
template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

// emplace_back / 线程池 submit / 装饰器 / 日志包装 都靠它
pool.submit([f = std::forward<F>(func)]() mutable { f(); });
```

记忆口诀：`std::move` 用在「我不要这个对象了」；`std::forward` 只用在「万能引用参数」上，且模板参数要写全 `std::forward<T>(arg)`。

1.5 统一初始化 `{}`

```
int          a{5};
double       b{1.5};
std::vector<int> v{1, 2, 3};
std::map<int, std::string> m{{1, "one"}, {2, "two"}};
struct P { int x, y; }; P p{1, 2};
```

好处 1：禁止窄化转换

```
int narrow{3.14}; // ❌ 编译错误
int old = 3.14;   // ⚠️ 只是警告，静默截断成 3
```

【坑】() 和 {} 对有 initializer_list 构造的类意义完全不同

```
std::vector<int> v1(10, 5);    // 10 个元素，都是 5
std::vector<int> v2{10, 5};    // 2 个元素：10 和 5
```

规则：只要类有 `initializer_list` 构造函数，`{}` 就会优先选它，哪怕其它构造函数更匹配。

实践建议：容器指定「个数」时用 `()`，指定「内容」时用 `{}`。其它情况一律用 `{}`。

1.6 Lambda 表达式

[捕获列表](参数列表) mutable noexcept -> 返回类型 { 函数体 }

```
int x = 10, y = 20;

auto f1 = [](int v) { return v * 2; };           // 无捕获（可转函数指针）
auto f2 = [x](int v) { return v + x; };          // 值捕获（副本，默认 const）
auto f3 = [&y](int v) { y += v; };               // 引用捕获
auto f4 = [=] { return x + y; };                // 全部按值
auto f5 = [&] { x++; y++; };                    // 全部按引用
auto f6 = [x]() mutable { x++; return x; };      // mutable 才能改值捕获的副本
auto f7 = [this] { return member_; };           // 捕获 this（注意生命周期！）
```

Lambda 的本质

编译器把 `[x](int a){ return a + x; }` 展开成：

```
class __某个匿名类 {
    int x;                                     // 捕获的变量变成成员
public:
    __某个匿名类(int x_) : x(x_) {}
    auto operator()(int a) const { return a + x; } // 默认 const，加 mutable 就去掉
};
```

推论：

- 无捕获的 lambda 可以隐式转成函数指针（因为不需要存状态），能传给 C API
- 有捕获的不行——它是个有状态的对象
- `mutable` 的作用就是把 `operator()` 的 `const` 去掉

【坑】捕获的生命周期

```
std::function<int()> make_bad() {  
    int local = 42;  
    return [&local] { return local; };    // ✗ 返回后 local 已销毁，悬垂引用  
}  
  
// 异步场景更危险  
void submit_task() {  
    std::string data = load();  
    pool.submit([&data] { process(data); }); // ✗ 函数返回时 data 就没了  
    pool.submit([data] { process(data); }); // ✓ 值捕获 (C++14 可以移动捕获)  
}
```

经验法则： - 立即同步使用（传给 `std::sort` 之类）→ `[&]` 没问题，最高效 - 要存起来、跨线程、异步执行 → 必须值捕获或移动捕获

捕获 `this` 尤其危险 —— 捕的是指针，对象死了 lambda 还活着就崩。C++17 起可以 `[*this]` 拷贝整个对象。

【用在哪】 STL 算法谓词、回调、局部小函数、延迟执行、线程任务。可以说现代 C++ 代码里 lambda 无处不在。

1.7 智能指针

`unique_ptr` —— 独占所有权，默认选它

```
auto p = std::make_unique<Widget>(1, 2);    // C++14; C++11 用 unique_ptr<W>(new W)  
p->method();  
auto q = std::move(p);                      // 转移所有权, p 变空  
// 不可拷贝，只能移动 — 语义上就保证了「只有一个所有者」
```

零开销： `sizeof(unique_ptr<T>) == sizeof(T*)`，编译后和裸指针一样快。

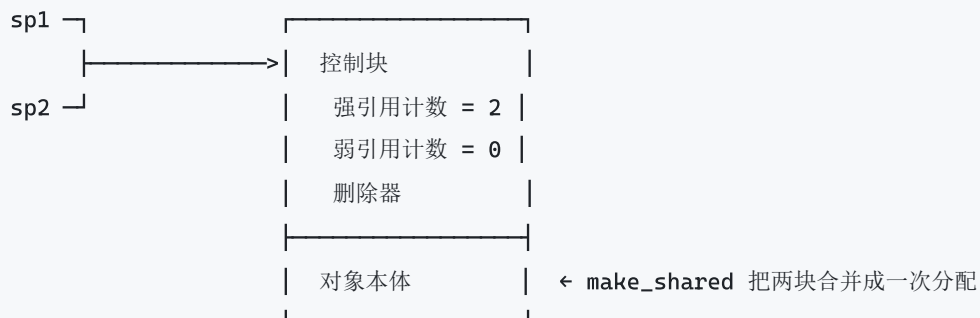
自定义删除器管 C 资源：

```
auto closer = [](FILE* f) { if (f) fclose(f); };  
std::unique_ptr<FILE, decltype(closer)> fp(fopen("a.txt", "r"), closer);
```

shared_ptr —— 引用计数共享

```
auto sp1 = std::make_shared<Widget>(); // 一次分配（对象+控制块），比 new 好
{
    auto sp2 = sp1;                      // 计数 2
}                                         // sp2 析构，计数 1
// 计数归 0 时才 delete 对象
```

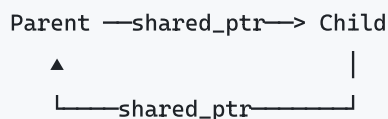
内存布局：



代价：控制块是额外分配；引用计数是**原子操作**（多线程安全但慢）；`sizeof(shared_ptr)` 是裸指针的两倍。

weak_ptr —— 打破循环引用

【面试】循环引用为什么泄漏



Parent 的计数 = 1（被 Child 持有）

Child 的计数 = 1（被 Parent 持有）

外部所有引用都释放后，两者计数都还是 1，永远不为 0 → 两个对象都不析构

修正：把**反向那条边**改成 `weak_ptr`：

```

struct Child;
struct Parent { std::shared_ptr<Child> child; };
struct Child { std::weak_ptr<Parent> parent; }; // ★ 弱引用, 不增加计数

// 使用时提升
if (auto p = child->parent.lock()) { // lock() 返回 shared_ptr, 可能为空
    p->do_something();
}

```

`weak_ptr` 的其它用途: 缓存 (对象没了自动失效)、观察者列表 (避免悬垂回调)。

三条经验

1. 默认 `unique_ptr`, 只有确实需要多方共享所有权才升级到 `shared_ptr`
2. 用 `make_unique` / `make_shared`, 不要裸 `new` (异常安全 + 少一次分配)
3. 函数参数怎么传: - 只是用一下, 不管生命周期 → `const T&` 或 `T*` - 要接管所有权 → `std::unique_ptr<T>` 按值传 - 要共享所有权 → `std::shared_ptr<T>` 按值传 - 只是想读 `shared_ptr` 指向的东西 → 还是传 `const T&`, 别传 `shared_ptr` (省一次原子操作)

1.8 变参模板

```

// C++11 靠递归展开
template <typename T>
void print(const T& t) { std::cout << t << "\n"; }

template <typename T, typename... Rest>
void print(const T& t, const Rest&... rest) {
    std::cout << t << " ";
    print(rest...); // 每次少一个参数
}

print(1, 2.5, "three", 'c');

```

C++17 有了折叠表达式就不用递归了 (见第 3 章)。

【用在哪】 `make_unique` / `emplace_back` / `std::tuple` / 类型安全的 `printf` / 事件系统 / 线程池 `submit`。

1.9 其它 C++11 特性速览

特性	写法	用在哪
<code>nullptr</code>	<code>int* p = nullptr;</code>	替代 <code>NULL</code> (<code>NULL</code> 就是整数 0, 会造成重载歧义)
<code>enum class</code>	<code>enum class Color : uint8_t { Red };</code>	强类型枚举, 不污染作用域、不隐式转 <code>int</code>
<code>constexpr</code>	<code>constexpr int f(int n){ return n*n; }</code>	编译期计算: 数组大小、模板参数、查找表
范围 for	<code>for (const auto& x : v)</code>	遍历一切有 <code>begin()/end()</code> 的东西
<code>= default</code>	<code>Foo() = default;</code>	显式要一个编译器生成的版本
<code>= delete</code>	<code>Foo(const Foo&) = delete;</code>	禁止某个操作, 报错信息清晰
<code>override</code>	<code>void f() override;</code>	编译器检查确实覆盖了虚函数
<code>final</code>	<code>void f() final; / class C final</code>	禁止继续覆盖/继承
委托构造	<code>Foo(int a) : Foo(a, 0) {}</code>	消除多个构造函数的重复代码
继承构造	<code>using Base::Base;</code>	派生类直接复用基类的所有构造
类型别名	<code>template<class T> using Vec = vector<T>;</code>	模板别名 (<code>typedef</code> 做不到)
<code>static_assert</code>	<code>static_assert(sizeof(int)>=4, "msg");</code>	编译期断言
<code>noexcept</code>	<code>void f() noexcept;</code>	承诺不抛异常 (移动构造必加)
原始字符串	<code>R"(C:\path\file)"</code>	正则、路径、JSON, 不用转义反斜杠
用户定义字面量	<code>5.0_km</code>	带单位的常量, 类型安全
<code>std::array</code>	<code>std::array<int,5> a{...}</code>	定长数组, 比 C 数组好用
<code>alignas/alignof</code>	<code>alignas(64) char buf[64];</code>	缓存行对齐, 避免伪共享
类内成员初始化	<code>int x = 42;</code> 写在类里	减少构造函数样板
<code>explicit operator bool</code>	<code>explicit operator bool() const</code>	支持 <code>if(obj)</code> 但禁止 <code>int i = obj</code>
<code><cstdint></code>	<code>int32_t / uint64_t</code>	写协议和二进制格式必用 (<code>int</code> 大小随平台变)

1.10 多线程

```
#include <thread>
#include <mutex>
#include <atomic>
#include <future>
#include <condition_variable>
```

基本用法

```
std::thread t([]{ work(); });
t.join();           // 等它结束; 或 t.detach() 让它自生自灭
// 【坑】两个都不调, thread 析构时会 std::terminate
```

互斥锁

```
std::mutex mtx;

{
    std::lock_guard<std::mutex> lk(mtx);    // RAII, 构造上锁, 析构解锁
    shared_data++;
}                                           // 自动解锁, 异常也安全

{
    std::unique_lock<std::mutex> lk(mtx);   // 更灵活: 可提前 unlock、可延迟上锁
    cv.wait(lk, pred);                     // 配合条件变量必须用它
}

std::scoped_lock lk(m1, m2, m3);          // C++17: 一次锁多个, 内部防死锁
```

【坑】永远不要手动 lock() / unlock()，中间任何一条 return 或异常都会漏解锁。

条件变量

```
std::mutex m;
std::condition_variable cv;
std::queue<Task> q;
bool done = false;

// 消费者
{
    std::unique_lock lk(m);
    cv.wait(lk, [&]{ return !q.empty() || done; }); // ★ 一定要带谓词
    // wait 会: 解锁 -> 睡眠 -> 被唤醒 -> 重新上锁 -> 检查谓词 -> 不满足继续睡
}

// 生产者
{ std::lock_guard lk(m); q.push(task); }
cv.notify_one();
```

为什么必须带谓词： 1. **虚假唤醒：** 操作系统可能在没有 notify 的情况下唤醒线程 2. **丢失通知：** notify 发生在 wait 之前就白发了，谓词能检测到「条件已经满足」

原子操作

```
std::atomic<int> counter{0};
counter.fetch_add(1, std::memory_order_relaxed);
counter++; // 默认 seq_cst
```

内存序（先记结论，深入以后再看）

内存序	保证	用在哪
relaxed	只保证这个变量本身原子	纯计数器（统计、引用计数递增）
acquire	之后的读写不会重排到它前面	读端，和 release 配对
release	之前的读写不会重排到它后面	写端，和 acquire 配对
acq_rel	读改写操作两头都管	fetch_add 等 RMW 操作
seq_cst	全局单一顺序（默认）	不确定就用它

经验： 不确定就用默认的 seq_cst 。无锁编程的收益通常远小于写错的代价。

异步

```
// async: 我要「结果」，不关心用几个线程
auto fut = std::async(std::launch::async, []{ return compute(); });
do_other_work();
int r = fut.get();           // 阻塞等结果（只能 get 一次）

// promise/future: 手动在一个线程给另一个线程投递结果
std::promise<int> prom;
auto f = prom.get_future();
std::thread([&prom]{ prom.set_value(42); }).detach();
f.get();
```

【坑】 `std::async` 不带 `launch::async` 时，实现可以选择「延迟执行」（直到 `get()` 才在当前线程跑）——想真并行就显式写 `std::launch::async`。

其它

```
thread_local int tls_var;           // 每个线程一份
std::once_flag flag;
std::call_once(flag, []{ init(); }); // 只执行一次，线程安全
std::this_thread::sleep_for(100ms);
std::thread::hardware_concurrency(); // CPU 核数（提示值，可能返回 0）
```

第 2 章 · C++14

► 对应程序： ch02_cpp14

小版本，但补的几个洞非常实用。

特性	解决了什么问题	【用在哪】
泛型 lambda	C++11 的 lambda 参数必须写死类型	一个比较器/转换器给多种类型用
初始化捕获	C++11 没法把 unique_ptr 捕获进 lambda	异步任务、线程池、移动大对象进闭包
返回类型推导	C++11 只有 lambda 能省返回类型	短小的工具函数
decltype(auto)	auto 会丢引用和 const	泛型转发函数的返回类型
变量模板	要写 Trait<T>::value	标准库的 _v 后缀就是它
放宽 constexpr	C++11 只能一条 return	编译器查找表、编译期算法
make_unique	C++11 漏了 (只有 make_shared)	所有 unique_ptr 的创建
二进制字面量/数字分隔符	0b1010'1100、1'000'000	位掩码、大常量可读性
异构查找	set<string>.find("literal") 会构造临时 string	高频查找的热点代码

2.1 泛型 lambda

```
auto plus = [](auto a, auto b) { return a + b; };
plus(1, 2);                // int
plus(1.5, 2.5);            // double
plus(std::string("a"), "b"); // string
```

原理：编译器生成的闭包类里， operator() 是成员函数模板。

实用例子 —— 一个 lambda 打印任何容器：

```

auto dump = [](const auto& c, const char* name) {
    std::cout << name << ": ";
    for (const auto& e : c) std::cout << e << " ";
    std::cout << "\n";
};
dump(std::vector<int>{1,2,3}, "vec");
dump(std::set<std::string>{"a","b"}, "set");

```

完美转发版：

```

auto invoker = [](auto&& f, auto&&... args) {
    return f(std::forward<decltype(args)>(args)...);
};

```

2.2 初始化捕获（移动捕获）

C++11 的痛点： `unique_ptr` 不能拷贝，所以根本没法捕获进 `lambda`。

```

auto up = std::make_unique<Data>();

auto task = [p = std::move(up)]() {      // ✅ 移动进闭包，闭包成为所有者
    return p->process();
};
// 此时 up 已为空

// 还能捕获「新算出来的值」
auto f = [n = expensive_compute(), msg = std::string("结果")] {
    std::cout << msg << n;
};

// 避免捕获 this 悬垂：只带走需要的成员
auto g = [name = this->name_] { use(name); };

```

2.3 decltype(auto)

```

struct Holder { std::vector<int> data; std::vector<int>& get() { return data; } };

auto          by_auto(Holder& h)      { return h.get(); }    // 返回 vector（拷贝！）
decltype(auto) by_decltype(Holder& h) { return h.get(); }    // 返回 vector&（引用）

```

规律： `auto` 按值语义推导（丢引用和顶层 `const`）； `decltype(auto)` 原样保留。

写泛型转发函数（不知道被转发的东西返回值还是引用）时，用 `decltype(auto)`。

2.4 放宽的 constexpr —— 编译期查找表

```
constexpr int kSize = 16;
struct SquareTable {
    int values[kSize]{};
    constexpr SquareTable() {                // C++11 里循环是非法的
        for (int i = 0; i < kSize; ++i) values[i] = i * i;
    }
};
constexpr SquareTable kSquares{};
static_assert(kSquares.values[7] == 49);
```

编译完这张表就躺在 `.rdata` 段里，运行期一次乘法都不用做。

【用在哪】 CRC 表、三角函数表、状态机跳转表、编译期字符串哈希、协议字段偏移表。

2.5 为什么必须用 `make_unique`

```
// ❌ 有泄漏风险
f(std::unique_ptr<A>(new A), g());
// 编译器可以按这个顺序求值: new A -> g() -> unique_ptr 构造
// 如果 g() 抛异常, new 出来的 A 永远不会被接管 → 泄漏

// ✅ 安全
f(std::make_unique<A>(), g());
// make_unique 内部把「分配 + 接管」打包成不可分割的一步
```

`make_shared` 额外好处：把控制块和对象合并成一次内存分配（性能更好，缓存更友好）。但注意：

`make_shared` 分配的内存要等所有 `weak_ptr` 都释放后才归还（因为控制块和对象在同一块内存里）——大对象 + 长期 `weak_ptr` 的场景要留意。

第 3 章 · C++17

► 对应程序：ch03_cpp17

没有 C++11 那么颠覆，但每一条都是日常代码里天天能用的。

3.1 结构化绑定

```
// 遍历 map —— 最常见的用途
for (const auto& [name, score] : scores) {
    std::cout << name << ": " << score << "\n";
}
// 对比 C++14: kv.first / kv.second, 可读性差远了

// 接收 pair 返回值
auto [it, inserted] = m.insert({"key", 1});

// 拆 tuple
auto [a, b, c] = std::tuple{1, 2.0, "x"};

// 拆结构体（公有成员，按声明顺序）
struct Point { int x, y; };
auto [px, py] = get_point();

// 绑引用可修改
for (auto& [k, v] : m) v *= 2;
```

限制：不能跳过成员、不能给绑定名加类型、不能用于有 private 成员的类。

3.2 if / switch 带初始化语句

```
if (auto it = m.find(key); it != m.end()) {
    use(it->second);
} else {
    // it 在 else 分支里也可见
}
// it 的作用域到这里结束 —— 不污染外层
```

【用在哪】 凡是「先取一个临时值，再判断」的地方。查找、加锁、打开文件、检查返回码。

3.3 CTAD 类模板参数推导

```
std::pair p{1, 2.0};           // 不用写 std::pair<int, double>
std::vector v{1, 2, 3};
std::lock_guard lk{mtx};       // 不用写 lock_guard<std::mutex>
std::array a{1, 2, 3};         // array<int, 3>

// 自定义推导指引
template <typename T> struct Wrapper { T value; };
Wrapper(const char*) -> Wrapper<std::string>; // 让 Wrapper("x") 推成 string 版
```

3.4 if constexpr —— 杀手级特性

```
template <typename T>
auto describe(const T& v) {
    if constexpr (std::is_pointer_v<T>) {
        return *v;           // T 不是指针时这段【根本不编译】
    } else if constexpr (std::is_integral_v<T>) {
        return v * 2;
    } else {
        return v;
    }
}
```

和普通 if 的本质区别：普通 if 的两个分支都必须能编译通过；if constexpr 中条件为假的分支不实例化，里面写什么都行。

对比 C++17 之前的 SFINAE 写法：

```
template <typename T>
std::enable_if_t<std::is_pointer_v<T>, std::remove_pointer_t<T>> f(T t) { return *t; }

template <typename T>
std::enable_if_t<!std::is_pointer_v<T>, T> f(T t) { return t; }
```

要写两个函数、语法晦涩、报错几百行。if constexpr 一举取代了它和 tag dispatch。

3.5 折叠表达式

```
template <typename... Ts> auto sum(Ts... ts)      { return (ts + ...); }
template <typename... Ts> auto sum0(Ts... ts)     { return (0 + ... + ts); } // 空包安全
template <typename... Ts> void print(Ts&&... ts)  { ((std::cout << ts << " "), ...); }
template <typename... Ts> bool all(Ts... ts)     { return (ts && ...); }

template <typename T, typename... Ts>
bool is_any_of(const T& v, const Ts&... cand) { return ((v == cand) || ...); }
```

四种形式：

(E op ...)	一元右折叠	E1 op (E2 op E3)
(... op E)	一元左折叠	(E1 op E2) op E3
(E op ... op I)	二元右折叠	带初值 I，空包时返回 I
(I op ... op E)	二元左折叠	带初值 I，空包时返回 I

3.6 std::optional

```
std::optional<int> parse(std::string_view s);

auto r = parse("42");
if (r)      use(*r);           // 像指针一样用
if (r.has_value()) use(r.value()); // value() 无值时抛 bad_optional_access
int v = r.value_or(-1);       // 给默认值
r.reset(); r.emplace(5);
```

为什么比「返回 -1 / 空指针 / 传出参数」好

1. **值语义**，不涉及所有权和生命周期
2. **类型上写明了「可能没有」**，调用方必须处理，编译器帮你记着
3. **不需要哨兵值**（不用约定 -1 表示失败，也就不怕 -1 是合法结果）

【用在哪】查找、解析、可选配置项、可能失败的构造。

【坑】 `sizeof(optional<T>) = sizeof(T) + 对齐后的一个 bool`，不是零成本。大对象放 optional 里要留意。

3.7 std::variant —— 类型安全的 union

```
std::variant<int, std::string, double> v = 42;
v = std::string("hello");

std::get<std::string>(v); // 类型不对会抛 bad_variant_access
if (auto* p = std::get_if<int>(&v)) { } // 安全版，不对返回 nullptr
std::holds_alternative<int>(v);
v.index(); // 当前是第几个类型
```

overloaded 惯用法 (值得背下来)

```
template <class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template <class... Ts> overloaded(Ts...) -> overloaded<Ts...>;

std::visit(overloaded{
    [](int i) { /* ... */ },
    [](const std::string& s) { /* ... */ },
    [](double d) { /* ... */ },
}, v);
```

【用在哪】 状态机、AST 节点、解析结果、消息类型、「要么成功要么错误」的返回值。

```
// 状态机：每个状态带自己的数据
struct Idle {};
struct Running { int progress; };
struct Done { std::string result; };
using State = std::variant<Idle, Running, Done>;
```

好处： 1. 不合法的状态**根本表示不出来**（不像 enum + 一堆散落的成员） 2. **漏处理一个分支**，visit 编译期就报错 3. 无堆分配，全在栈上

3.8 `std::string_view` —— 投入产出比最高的性能优化

```
void log(std::string_view sv);           // 一个参数类型，接受所有字符串

log("字面量");                          // 零分配
log(std::string("x"));                   // 零拷贝
log(some_string.substr(0, 5));           // 注意: string::substr 还是会分配

std::string_view sv = "hello world";
auto sub = sv.substr(0, 5);              // string_view::substr 是 O(1)，只挪指针
```

内部就是 `{const char* ptr; size_t len;}`，拷贝极便宜。

三个必须记住的坑

```
// ① 不拥有数据 —— 悬垂
std::string_view bad() {
    std::string s = "temp";
    return s;                               // ❌ 返回后 s 销毁，view 悬垂
}

std::string_view bad2 = get_string().substr(0, 3); // ❌ 临时 string 立刻销毁

// ② 不保证 '\0' 结尾 —— 不能直接给 C API
printf("%s", sv.data());                   // ❌ 可能读越界
printf("%.s", (int)sv.size(), sv.data()); // ✅
std::string(sv).c_str();                   // ✅ (有一次分配)

// ③ 别做成员变量，除非能保证被指向的字符串活得更久
```

规则: `string_view` 只做函数参数和短命的局部变量。

3.9 std::filesystem

```
namespace fs = std::filesystem;

fs::path p = fs::temp_directory_path() / "mydir";    // / 运算符拼路径，跨平台
fs::create_directories(p / "sub");
fs::exists(p); fs::is_directory(p); fs::file_size(f);

p.filename(); p.stem(); p.extension(); p.parent_path();

for (const auto& e : fs::recursive_directory_iterator(p)) {
    std::cout << e.path() << " " << (e.is_regular_file() ? e.file_size() : 0) << "\n";
}

fs::copy(src, dst, fs::copy_options::recursive);
fs::remove_all(p);
```

不用再为 Windows 的 \ 和 Linux 的 / 写两套代码。

3.10 并行算法

```
#include <execution>
std::sort(std::execution::par, v.begin(), v.end());
std::for_each(std::execution::par_unseq, v.begin(), v.end(), f);
```

策略	含义
seq	串行（等同不带策略）
par	多线程并行
par_unseq	并行 + 向量化（元素间不能有任何依赖，不能加锁）
unseq	仅向量化（C++20）

【坑】 - 小数据用 par 反而更慢（线程开销） - 传给它的函数**不能抛异常、不能加锁、不能有数据竞争** - GCC 需要链接 TBB 才能真正并行

std::reduce vs std::accumulate : accumulate 保证从左到右依次累加，所以**无法并行**； reduce 不保证顺序，可以并行，但要求运算满足**结合律和交换律**。浮点加法不满足结合律，所以两者的浮点结果可能有微小差异。

3.11 其它 C++17 实用点

```
// inline 变量: 头文件里定义全局变量不再冲突
inline constexpr int kMaxRetries = 3;
struct S { static inline int count = 0; }; // 类内静态成员就地定义

// 嵌套命名空间
namespace a::b::c { }

// 保证的拷贝省略: 不可移动不可拷贝的类型也能从函数返回了
NonMovable make() { return NonMovable(1); }

// 属性
[[nodiscard]] int must_check(); // 忽略返回值就警告
void f([[maybe_unused]] int x);
switch (n) { case 1: a(); [[fallthrough]]; case 2: b(); }

// 新工具
std::clamp(v, lo, hi); std::gcd(a,b); std::lcm(a,b);
std::apply(func, tuple); // tuple 展开成参数
std::invoke(callable, args...); // 统一调用语法
std::byte b{0xFF}; // 真正的字节类型 (不是 char)

// from_chars / to_chars — 最快的数值转换
int val;
auto [ptr, ec] = std::from_chars(s.data(), s.data()+s.size(), val);
// 不分配内存、不看 locale、不抛异常, 比 stoi/sprintf 快数倍
// 【用在哪】日志、JSON 序列化、协议解析等热点路径

// map / set 新接口
m.try_emplace(key, args...); // 键存在就【不构造 value】
m.insert_or_assign(key, val);
auto node = m.extract(key); // 摘出节点, 可改键后放回 (零拷贝)
m1.merge(m2);
```

【坑】 `m[key]` 在键不存在时会默认构造并插入。只读查询用 `find / at / contains`。

第 4 章 · C++20

► 对应程序：ch04_cpp20

四大特性俗称 "Big Four": **Concepts / Ranges / Coroutines / Modules**。

4.1 Concepts —— 让模板报错变成人话

定义

```
#include <concepts>

// 组合已有概念
template <typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;

// requires 表达式: 描述「这个类型必须支持哪些操作」
template <typename T>
concept Container = requires(T c) {
    typename T::value_type;           // 类型要求
    c.begin();                        // 简单要求
    { c.size() } -> std::convertible_to<size_t>; // 复合要求: 合法 + 返回类型
    requires std::copyable<T>;       // 嵌套要求
};
```

四种用法 (效果完全相同)

```
template <Numeric T> T f1(T v);           // 模板参数位置
template <typename T> requires Numeric<T> T f2(T v); // requires 子句
template <typename T> T f3(T v) requires Numeric<T>; // 尾置 requires
auto f4(Numeric auto v);                 // 缩写函数模板 (最简洁)
```

为什么重要

```
// C++17: 传错类型, 报错 200 行, 全是标准库内部的实例化栈
std::sort(list.begin(), list.end()); // list 迭代器不是随机访问

// C++20: 报错一行
// error: 'std::list<int>::iterator' does not satisfy 'random_access_iterator'
```

还能参与**重载决议**（更特化的概念优先）：

```
template <std::integral T> void f(T); // 传 int 走这个
template <std::floating_point T> void f(T); // 传 double 走这个
```

常用标准概念： integral floating_point same_as convertible_to derived_from
equality_comparable totally_ordered copyable movable invocable predicate ranges::range
ranges::sized_range

【**用在哪**】 所有对外暴露的模板接口。约束写清楚 = 文档 + 编译期检查 + 好报错。

4.2 Ranges —— 管道式算法

对比

```
std::vector<int> v{1,2,3,4,5,6,7,8,9,10};

// 传统: 三段代码 + 两个中间容器
std::vector<int> evens, squares;
std::copy_if(v.begin(), v.end(), std::back_inserter(evens), is_even);
std::transform(evens.begin(), evens.end(), std::back_inserter(squares), sq);
squares.resize(3);

// Ranges: 一行, 零中间容器, 惰性求值
auto result = v | std::views::filter(is_even)
               | std::views::transform(sq)
               | std::views::take(3);
```

惰性求值： 不遍历就一次都不算。 ch04_cpp20 里实测：建好管道后计数器还是 0，取第一个元素才算了一次。

算法直接吃容器

```
std::ranges::sort(v); // 不用 v.begin(), v.end()
std::ranges::find(v, 5);
std::ranges::count_if(v, pred);

// 投影: 按对象的某个成员操作 — 极其好用
std::ranges::sort(people, {}, &Person::age); // 按 age 排序
std::ranges::find(people, 30, &Person::age); // 找 age==30 的
std::ranges::max_element(people, {}, &Person::score);
```

常用 views

view	作用
<code>filter(pred)</code>	过滤
<code>transform(f)</code>	变换
<code>take(n)</code> / <code>drop(n)</code>	取前 n / 跳过前 n
<code>take_while(p)</code> / <code>drop_while(p)</code>	条件版
<code>reverse</code>	反向
<code>iota(1, 10)</code> / <code>iota(1)</code>	生成序列 (后者无限)
<code>join</code>	摊平嵌套容器
<code>split(delim)</code>	分割
<code>keys</code> / <code>values</code>	map 的键/值视图
<code>elements<N></code>	tuple 容器取第 N 个

管道可以命名和复用:

```
auto even_squares = std::views::filter(is_even) | std::views::transform(sq);
auto r1 = v1 | even_squares;
auto r2 = v2 | even_squares;
```

【坑】视图不拥有数据。 `auto v = get_vector() | views::filter(p);` 里临时 vector 立刻销毁，视图悬挂。

4.3 协程

三个关键字: `co_await`、`co_yield`、`co_return`。函数体里出现任意一个，它就是协程。

执行流程

调用协程

- |
- |→ 在堆上分配 `coroutine frame` (保存局部变量 + 恢复点)
- |→ 构造 `promise_type`
- |→ `promise.get_return_object()` → 返回给调用者
- |→ `promise.initial_suspend()` → 决定「立刻执行」还是「先挂起」
- |

▼

函数体开始执行

- |
- |→ 遇到 `co_yield v`
 - |→ `promise.yield_value(v)` → 挂起, 控制权还给调用者
- |
- |→ (调用者调 `handle.resume()`)
 - |→ 从上次挂起点继续
- |
- |→ 遇到 `co_await awaitable`
 - |→ `awaiter.await_ready()` 为 `true` 就不挂起
 - |→ `awaiter.await_suspend()` 保存状态, 交出控制权
 - |→ `awaiter.await_resume()` 恢复后拿结果
- |

▼

`co_return` / 函数结束

- |→ `promise.return_value()` / `return_void()`
- |→ `promise.final_suspend()`
- |→ `handle.destroy()` 释放 `frame`

标准只给了**底层机制**, 没给现成的库。 `ch04_cpp20` 里手写了一个最小 Generator (约 30 行), 能这样用:

```
Generator<int> fibonacci(int n) {  
    int a = 0, b = 1;  
    for (int i = 0; i < n; ++i) {  
        co_yield a;  
        int t = a + b; a = b; b = t;  
    }  
}  
  
for (int x : fibonacci(10)) std::cout << x << " ";    // 0 1 1 2 3 5 8 13 21 34
```

【用在哪】 - **生成器**: 惰性产生序列, 不用一次性构造整个 `vector` (大文件按行读、无限序列) - **异步 IO**: 把回调地狱变回线性代码 —— 这是协程最大的价值 - **状态机**: 把状态隐含在「执行到哪一行」里, 不用手写 `switch`

现状: 业务里一般用 `cppcoro`、`asio` 的协程支持、或 C++23 的 `std::generator`, 很少自己写 `promise_type`。

4.4 三向比较 <=>

```
struct Point {
    int x, y;
    auto operator<=>(const Point&) const = default;    // 一行生成 < <= > >=
    bool operator==(const Point&) const = default;    // == 和 != 要单独 default
};
```

= default 的行为：按**成员声明顺序**逐个比较（字典序），第一个不相等的决定结果。

手写版：

```
struct Version {
    int major, minor, patch;
    std::strong_ordering operator<=>(const Version& o) const {
        if (auto c = major <=> o.major; c != 0) return c;
        if (auto c = minor <=> o.minor; c != 0) return c;
        return patch <=> o.patch;
    }
};
```

三种比较类别：

类别	含义	例子
strong_ordering	等价即可互相替换	int, string
weak_ordering	等价但不完全相同	忽略大小写的字符串
partial_ordering	可能 不可比较	浮点（NaN 和谁都不可比）

4.5 std::span

```
void process(std::span<const int> s);    // 一个参数接受所有连续容器

std::vector<int> v; int arr[3]; std::array<int,2> a;
process(v); process(arr); process(a); process({ptr, n});

s.subspan(1, 2); s.first(3); s.last(2);
s.size(); s.size_bytes();
```

对比老写法 void f(int* p, size_t n) —— span 把两个参数绑成一个，不会出现「指针和长度对不上」的 bug。

和 `string_view` 一样：不拥有数据，注意生命周期。

4.6 `std::format`

```
std::format("{} + {} = {}", 1, 2, 3);
std::format("[{:>10}]", "右对齐");           // [      右对齐]
std::format("[{: ^10}]", "居中");
std::format("{:.3f}", 3.1415926);             // 3.142
std::format("{: #x} {: #b} {: #o}", 255, 5, 8);
std::format("{:08.2f}", 3.1);                 // 00003.10
std::format("{1} {0}", "a", "b");             // 位置参数
std::format("{: {}}", "x", width);            // 动态宽度
std::format("{: +d}", 5);                     // +5
```

	printf	iostream	format
类型安全	✗ %d 配 double 是 UB	✓	✓
简洁	✓	✗ 极啰嗦	✓
编译器检查格式串	✗	N/A	✓
速度	中	慢	最快
全局状态污染	无	✗ <code>setw</code> 等是流状态	无

自定义类型：

```
template <> struct std::formatter<Point> {
    constexpr auto parse(auto& ctx) { return ctx.begin(); }
    auto format(const Point& p, auto& ctx) const {
        return std::format_to(ctx.out(), "({},{})", p.x, p.y);
    }
};
```

4.7 并发新工具

```
// jthread: 自动 join + 可取消
{
    std::jthread jt[](std::stop_token st) {
        while (!st.stop_requested()) work();
    };
} // 析构自动 request_stop() + join()

// latch: 一次性倒数门闩
std::latch done{3};
// 各线程: done.count_down();
done.wait();

// barrier: 可重用同步点 (分阶段计算)
std::barrier bar{4, []() noexcept { std::cout << "本阶段完成\n"; }};
// 各线程: bar.arrive_and_wait();    ← 注意完成回调必须 noexcept

// semaphore: 限流
std::counting_semaphore<10> sem{3};
sem.acquire(); /* 临界区 */ sem.release();

// atomic 的 wait/notify: 不用条件变量的轻量等待
std::atomic<int> flag{0};
flag.wait(0); // 阻塞直到值不再是 0
flag.store(1); flag.notify_all();

std::atomic_ref<int> ar{plain_int}; // 给非原子对象套原子访问
```

4.8 其它 C++20

```
// 指定初始化器（比简单 Builder 轻量得多）
struct Config { int width = 800; int height = 600; bool fs = false; };
Config c{.width = 1920, .height = 1080};           // 必须按声明顺序，可跳过

// using enum
enum class Color { Red, Green };
{ using enum Color; Color c = Red; }

// consteval / constexpr
constexpr int must_ct(int n) { return n * n; } // 只能编译期调用
constexpr int g = must_ct(3);                 // 保证静态初始化，避开初始化顺序灾难

// 模板 lambda
auto f = [<typename T>(const std::vector<T>& v) -> T { return v.front(); }];

// 位操作 <bit>
std::popcount(x);      std::countl_zero(x);   std::countr_zero(x);
std::has_single_bit(x); std::bit_ceil(x);     std::bit_width(x);
std::rotr(x, 3);       std::bit_cast<float>(bits);
std::endian::native == std::endian::little;

// 数学常量
std::numbers::pi; std::numbers::e; std::numbers::sqrt2;

// source_location（替代 __FILE__ / __LINE__ 宏）
void log(std::string_view msg,
         std::source_location loc = std::source_location::current()) {
    std::cout << loc.file_name() << ":" << loc.line()
               << " " << loc.function_name() << ": " << msg;
}

// 日历与时区
auto today = std::chrono::year_month_day{
    std::chrono::floor<std::chrono::days>(std::chrono::system_clock::now())};
auto d = 2026y / std::chrono::September / 7;
std::chrono::zoned_time zt{"Asia/Shanghai", std::chrono::system_clock::now()};

// 容器新接口
m.contains(key);   s.starts_with("pre");   s.ends_with(".txt");
std::erase_if(v, pred);           // 取代 erase-remove 惯用法
std::ssize(v);                 // 有符号长度，避免有符号比较警告
```

```
// 分支提示（只在真正的热点用）
```

```
if (ptr) [[likely]] { fast(); } else [[unlikely]] { slow(); }
```

4.9 Modules（了解即可）

```
// math.ixx
```

```
export module math;
```

```
export int add(int a, int b) { return a + b; }
```

```
int helper() { return 0; }
```

```
// 不导出 = 外部完全看不见
```

```
// 使用
```

```
import math;
```

好处：无宏泄漏、无重复解析、编译更快、真正的封装。**现状**（2026）：MSVC 支持最好，GCC/Clang 追赶中，构建系统集成仍然麻烦。新项目可以尝试，老项目别急着迁。

第 5 章 · C++23

► 对应程序：`ch05_cpp23`（该程序会自动检测编译器支持情况，不支持的特性打印提示而不是编译失败）
主题是「把 C++20 没做完的做完」。

5.1 `std::expected` —— 带错误值的返回

```
enum class ParseError { Empty, NotANumber, OutOfRange };

std::expected<int, ParseError> parse_port(std::string_view sv) {
    if (sv.empty()) return std::unexpected(ParseError::Empty);
    // ...
    return value;
}

auto r = parse_port("8080");
if (r) use(*r);
else    handle(r.error());
int v = r.value_or(-1);

// 单子式接口：链式处理，不用层层 if
auto result = parse_port("8080")
    .transform([](int p) { return p + 1; }) // 成功时变换
    .and_then(next_step)                  // 继续可能失败的操作
    .or_else(fallback);                   // 失败时兜底
```

三种错误处理方式的取舍

	异常	错误码	<code>expected</code>
污染返回类型	否	是	是
调用方能忽略	否（会崩）	是（危险）	否（编译器逼你处理）
携带额外信息	是	否	是
性能	抛出时有栈展开开销	最快	快（无动态分配）
签名可见性	✗ 看不出会不会抛	✓	✓

经验：可预期的失败（解析、查找、IO、网络）用 `expected`；真正的异常情况（内存耗尽、不变量被破坏、程序 bug）用异常。

C++23 也给 `std::optional` 补了 `and_then` / `transform` / `or_else`。

5.2 `std::print` / `println`

```
std::println("{} items in {:.2f}s", n, secs);
std::print(stderr, "error: {}\n", msg);
```

比 `std::cout << std::format(...)` 更短，实现上直接写 `stdout`，更快。

5.3 Ranges 补全

```
namespace rv = std::views;

auto vec = r | std::ranges::to<std::vector>(); // 视图 → 容器（终于有了）

for (auto [i, x] : v | rv::enumerate) { } // 带索引遍历
for (auto [a, b] : rv::zip(va, vb)) { } // 并行遍历多个容器
v | rv::chunk(3); // 分块
v | rv::slide(2); // 滑动窗口
v | rv::stride(2); // 隔 n 取一个
v | rv::chunk_by(pred); // 按条件分组
v | rv::join_with(", "); // 拼接
v | rv::adjacent<2>; // 相邻元素成对
rv::cartesian_product(a, b);
```

5.4 `deducing this` —— 显式对象参数

```
struct Container {
    std::vector<int> data;

    template <typename Self>
    auto&& items(this Self&& self) { return std::forward<Self>(self).data; }
    // 一个函数模板同时充当 T&、const T&、T&& 四个重载
};

// 递归 lambda 一行搞定（以前要 std::function 或 Y 组合子）
auto fac = [](this auto self, int n) -> int { return n <= 1 ? 1 : n * self(n - 1); };
```

5.5 其它

```
std::generator<int> range(int n) { for (int i=0;i<n;++i) co_yield i; } // 标准生成器

std::mdspan m(data.data(), 3, 4); m[1, 2] = 5.0f; // 多维视图 + 多维下标

std::flat_map<int, std::string> fm; // 底层是排序 vector, 缓存友好
// 优点: 遍历/查找快, 内存省 缺点: 插入删除 O(n), 迭代器易失效
// 【用在哪】建好之后基本只读、元素不多 (几百个内) 的映射表

if constexpr { compile_time_path(); } else { runtime_path(); }

struct Less { static bool operator()(int a, int b) { return a < b; } }; // 静态 operator()

std::to_underlying(enum_val); // enum → 底层类型
std::byteswap(x); // 大小端翻转
s.contains("sub"); // string 也有 contains 了
std::stacktrace::current(); // 标准调用栈
std::unreachable(); // 优化提示
std::move_only_function<void()> // 能装 move-only 的 std::function
import std; // 标准库模块
```

5.6 C++26 展望 (已定案)

- **反射**: `^^Type` 拿到元信息, 编译期遍历成员 —— 序列化再也不用写宏了
- **契约**: `pre(x > 0)` / `post(r != nullptr)` / `contract_assert`
- **std::execution**: sender/receiver 异步框架, 标准化的结构化并发
- **std::hive**: 高性能稳定引用容器 (游戏引擎常用)
- **std::inplace_vector**: 固定容量、栈上分配的 vector

第 6 章 · 标准库 (STL)

► 对应程序: `ch06_stl`

6.1 容器选型决策树

需要按【键】查找吗？

- |
 - └ 是 → 需要【有序遍历 / 范围查询 / 前驱后继】吗？
 - |
 - └ 是 → `std::map` / `std::set`
红黑树, $O(\log n)$, 迭代器稳定, 遍历有序
 - └ 否 → `std::unordered_map` / `std::unordered_set`
哈希表, 均摊 $O(1)$, 顺序不确定, `rehash` 时迭代器失效
 - └ 否 → 大小编译期已知？
 - └ 是 → `std::array` 栈上, 零开销
 - └ 否 → 只在【尾部】增删？
 - └ 是 → `std::vector` ← 默认就选它
 - └ 否 → 【两端】都增删？
 - └ 是 → `std::deque`
 - └ 否 → 需要迭代器永久有效 或 $O(1)$ `splice`？
 - └ 是 → `std::list`
 - └ 否 → 还是 `vector` (实测通常更快)

最重要的一条经验：不确定就用 `vector`。

链表的理论优势 ($O(1)$ 中间插入) 在现代 CPU 上常被缓存不命中吃光。遍历 `vector` 比遍历 `list` 快 5~10 倍是常见现象, 因为 `vector` 内存连续、CPU 预取器能提前把数据拉进缓存, 而链表每跳一个节点就可能是一次缓存未命中 (约 100 个时钟周期)。

6.2 复杂度与性质速查

容器	随机访问	头插	尾插	中间插	查找	内存	迭代器失效
vector	O(1)	O(n)	均摊 O(1)	O(n)	O(n)	连续	扩容时全失效
deque	O(1)	O(1)	O(1)	O(n)	O(n)	分段连续	插入时迭代器失效，引用不失效
list	X	O(1)	O(1)	O(1)	O(n)	节点	只有被删的失效
array	O(1)	X	X	X	O(n)	栈上连续	不失效
map/set	X	—	—	O(log n)	O(log n)	红黑树节点	只有被删的失效
unordered_*	X	—	—	均摊 O(1)	均摊 O(1)	桶+链	rehash 时全失效

6.3 vector 的几个关键点

```
v.reserve(n);           // ★ 预分配，避免反复扩容 — 最容易拿到的性能提升
v.emplace_back(a, b);    // 原位构造，比 push_back(T(a,b)) 少一次移动
v.shrink_to_fit();       // 释放多余容量（非强制）
```

ch06_stl 实测：20 万次 push_back，不 reserve vs reserve 差距明显。

【坑】扩容会让所有迭代器/指针/引用失效

```
int* p = &v[0];
v.push_back(x);           // 可能重新分配
*p = 1;                   // ✗ p 可能已经悬垂
```

删除元素

```
std::erase_if(v, pred);           // C++20，一行
v.erase(std::remove_if(v.begin(), v.end(), pred), v.end()); // C++17 及以前

// 无序容器的 O(1) 删除技巧（不保序）
std::swap(v[i], v.back());
v.pop_back();
```

注意 `std::remove_if` 本身不删元素，它只是把要保留的元素挪到前面，返回新的逻辑末尾 —— 必须配合 `erase`。这就是 "erase-remove 惯用法"。

6.4 map 的坑与技巧

```
// ❌ operator[] 在键不存在时会【插入】一个默认构造的元素
if (m["key"] == 0) { }      // 这一句就把 "key" 插进去了

// ✅ 只读查询
m.contains(key);            // C++20
m.find(key) != m.end();
m.at(key);                  // 不存在抛 out_of_range

// 四种插入的区别
m[k] = v;                   // 不存在则默认构造 value 再赋值（两步）
m.insert({k, v});           // 已存在则什么都不做（但 value 已经构造好了）
m.try_emplace(k, args...);  // 已存在则【不构造 value】← 最省
m.insert_or_assign(k, v);   // 存在就覆盖
```

范围查询是 map 相对 unordered_map 的独门优势

```
auto lo = m.lower_bound(150);    // 第一个 >= 150
auto hi = m.upper_bound(350);    // 第一个 > 350
for (auto it = lo; it != hi; ++it) { }
```

自定义类型做 key

```
// map/set 需要【严格弱序】
struct Cmp { bool operator()(const K& a, const K& b) const { return a.id < b.id; } };
std::set<K, Cmp> s;

// unordered_* 需要 hash + operator==
struct Hash {
    size_t operator()(const K& k) const noexcept {
        size_t h1 = std::hash<std::string>{}(k.name);
        size_t h2 = std::hash<int>{}(k.age);
        return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2)); // hash_combine
    }
};
std::unordered_set<K, Hash> us;
```

6.5 算法速查

```
// — 排序 —
sort(b, e)                不稳定,  $O(n \log n)$ 
stable_sort(b, e)         稳定, 需要额外内存
partial_sort(b, mid, e)   只保证前 (mid-b) 个有序
nth_element(b, nth, e)    只保证第 n 个就位, 左边都不大于它,  $O(n)$  ← 求中位数
is_sorted / reverse / rotate / shuffle

// — 有序区间查找  $O(\log n)$  — ★ 前提: 必须已排序
binary_search(b, e, v)    只返回 bool
lower_bound(b, e, v)      第一个  $\geq v$ 
upper_bound(b, e, v)      第一个  $> v$ 
equal_range(b, e, v)      等于 v 的区间

// — 线性查找  $O(n)$  —
find / find_if / find_first_of / search / count / count_if
min_element / max_element / minmax_element
all_of / any_of / none_of

// — 修改 —
copy / copy_if / transform / fill / generate / replace
unique                    去掉【相邻】重复 (先 sort)
remove_if                 只移动不删除, 配合 erase
rotate / reverse

// — 划分 —
partition / stable_partition / partition_point

// — 集合运算 — ★ 两边都必须有序
set_union / set_intersection / set_difference / includes / merge

// — 数值 <numeric> —
accumulate(b, e, init)    严格从左到右, 不能并行
reduce(b, e, init)         顺序不保证, 可并行 (要求结合律)
inner_product(b1, e1, b2, init) 点积
transform_reduce(b, e, init, op, f) map-reduce
partial_sum / inclusive_scan 前缀和
adjacent_difference        相邻差分
iota(b, e, start)          填 start, start+1, ...
gcd / lcm / midpoint / lerp

// — 堆 —
make_heap / push_heap / pop_heap / sort_heap
```

6.6 字符串

```
// 拼接：性能差 10 倍以上
std::string r;
for (...) r = r + "x";           // ❌ 每次都产生新字符串
r.reserve(n);
for (...) r += "x";             // ✅ 原地追加

// 数值转换：三个档次
std::to_string(42) / std::stoi(s)    // 方便，慢，会抛异常，看 locale
std::ostringstream / istream        // 灵活，最慢
std::to_chars / std::from_chars      // ★ 最快：不分配、不抛、不看 locale
```

热点路径（日志、JSON、协议解析）一律用 `to_chars` / `from_chars`。

```
char buf[32];
auto [ptr, ec] = std::to_chars(buf, buf + 32, 12345);
std::string s(buf, ptr);

int v;
auto [p2, ec2] = std::from_chars(sv.data(), sv.data() + sv.size(), v);
if (ec2 != std::errc{}) { /* 解析失败 */ }
```

分割字符串（标准库没直接给）：

```
std::vector<std::string_view> split(std::string_view sv, char delim) {
    std::vector<std::string_view> out;
    size_t start = 0;
    while (start <= sv.size()) {
        size_t pos = sv.find(delim, start);
        if (pos == std::string_view::npos) { out.push_back(sv.substr(start)); break; }
        out.push_back(sv.substr(start, pos - start));
        start = pos + 1;
    }
    return out;
}
```

正则 <regex>：功能全，但**非常慢**（比其它语言的实现慢一个数量级），且编译时间长。热路径别用，简单匹配手写更快。

6.7 时间 <chrono>

```
using namespace std::chrono;
using namespace std::chrono_literals;

auto total = 1h + 30min + 45s;           // 类型安全，自动换算单位
duration_cast<seconds>(total).count();
duration<double>(total).count();

// ★ 测耗时一定用 steady_clock（单调递增，不受系统时间调整影响）
auto t0 = steady_clock::now();
work();
auto us = duration_cast<microseconds>(steady_clock::now() - t0).count();

// 显示日期用 system_clock（挂钟时间，可被 NTP 调整）
auto now = system_clock::now();
```

时钟	特性	用途
steady_clock	单调，绝不回退	测耗时、超时
system_clock	挂钟时间，可调整	显示日期、时间戳
high_resolution_clock	通常是上面某一个的别名	不推荐直接用

6.8 随机数 <random>

不要用 rand()：周期短、低位不随机、rand() % n 有偏差、线程不安全。

```
std::random_device rd;           // 熵源（可能很慢，只用来播种）
std::mt19937 gen(rd());          // 引擎：产生均匀比特流

std::uniform_int_distribution<int>    dice(1, 6);      // 分布：把比特流塑形
std::uniform_real_distribution<double> unit(0.0, 1.0);
std::normal_distribution<double>      gauss(0.0, 1.0);
std::discrete_distribution<>          weighted({10, 30, 60}); // 加权

int roll = dice(gen);
std::shuffle(v.begin(), v.end(), gen);

// 多线程：每线程一个引擎，别共享
thread_local std::mt19937 tls_gen{std::random_device{}};
```

架构要点：引擎和分布是分开的。 引擎产生原始随机比特，分布负责把它变成 你要的形状。这样任意引擎可以配任意分布。

6.9 函数对象 <functional>

```
std::function<int(int,int)> f = [](int a, int b) { return a + b; };
```

代价：可能有堆分配（超过小对象优化容量时）+ 间接调用 + **无法内联**。热点循环里慎用，那里应该用模板参数或直接传 lambda。

【用在哪】 回调注册表、事件总线、命令队列、任何「存起来以后调用」的场合。

```
// std::bind 已过时，一律用 lambda
auto add5_old = std::bind(add, 5, std::placeholders::_1); // ✗ 慢、难读、报错恐怖
auto add5_new = [](int b) { return add(5, b); }; // ✓

std::invoke(&S::method, obj, args...); // 统一调用语法（成员函数/成员变量/普通可调用）
std::ref(x) // 把引用放进容器或传给按值取参的模板
std::greater<>{} // 透明比较器，<> 表示类型自动推导
```

6.10 智能指针选型（再强调一次）

	大小	开销	用在哪
unique_ptr	8 字节（同裸指针）	零	默认选它
shared_ptr	16 字节	控制块 + 原子引用计数	确实需要共享所有权
weak_ptr	16 字节	同上	打破循环引用、缓存、观察者

函数参数怎么传

```
void f(const Widget& w); // 只是用一下 ← 最常见
void f(Widget* w); // 只是用一下，且可以为空
void f(std::unique_ptr<Widget> w); // 接管所有权（调用方必须 std::move）
void f(std::shared_ptr<Widget> w); // 共享所有权
void f(const std::shared_ptr<W>& w); // 想读 shared_ptr 本身（少见）
```

不要为了「看起来现代」就到处传 shared_ptr —— 每次拷贝都是一对原子操作。

第 7 章 · 设计模式（现代 C++ 版）

► 对应程序：ch07_patterns

先说结论：GoF 那 23 个模式是在 1994 年、面向 C++98/Java 总结的。现代 C++ 里有一批模式已经被语言特性直接吃掉了。这一章讲**现在还值得用的**，以及**每个模式的适用边界**。

C++ 专属惯用法（比 GoF 更常用，优先掌握）

RAII · Pimpl · CRTP · 类型擦除 · 策略模板

创建型 单例 工厂 建造者 原型

结构型 适配器 装饰器 代理 外观 桥接 组合 享元

行为型 策略 观察者 命令 模板方法 状态 访问者 责任链 迭代器 中介者 备忘录

7.1 RAII —— C++ 最重要的模式

资源获取即初始化：构造函数拿资源，析构函数还资源。因为析构**一定会**被调用（正常返回、提前 return、抛异常都一样），所以永不泄漏。

```
class FileHandle {
    std::FILE* f_ = nullptr;
public:
    FileHandle(const char* p, const char* m) : f_(std::fopen(p, m)) {
        if (!f_) throw std::runtime_error("打不开");
    }
    ~FileHandle() { if (f_) std::fclose(f_); }

    FileHandle(FileHandle&& o) noexcept : f_(std::exchange(o.f_, nullptr)) {}
    FileHandle& operator=(FileHandle&& o) noexcept {
        if (this != &o) { if (f_) std::fclose(f_); f_ = std::exchange(o.f_, nullptr); }
        return *this;
    }
    FileHandle(const FileHandle&) = delete; // 资源类通常禁止拷贝
    FileHandle& operator=(const FileHandle&) = delete;
};
```

通用 ScopeGuard

```

template <typename F>
class ScopeGuard {
    F f_; bool active_ = true;
public:
    explicit ScopeGuard(F f) : f_(std::move(f)) {}
    ~ScopeGuard() { if (active_) f_(); }
    void dismiss() { active_ = false; } // 成功了就取消回滚
};

template <typename F> ScopeGuard(F) -> ScopeGuard<F>;

// 用法: 事务回滚
void transfer() {
    begin_transaction();
    ScopeGuard rollback{[]{ rollback_transaction(); }};
    do_work(); // 抛异常就自动回滚
    commit();
    rollback.dismiss(); // 成功了, 取消回滚
}

```

【用在哪】 文件、锁、socket、数据库连接、GPU 句柄、事务、临时状态恢复、性能计时器。标准库里的例子: `unique_ptr` / `lock_guard` / `ifstream` / `jthread`。

7.2 Pimpl —— 隔断编译依赖

```
// widget.h — 干净, 不 #include 任何重量级依赖
class Widget {
public:
    Widget();
    ~Widget(); // ★ 必须在 .cpp 定义
    Widget(Widget&&) noexcept;
    Widget& operator=(Widget&&) noexcept;
    void do_work();
private:
    struct Impl;
    std::unique_ptr<Impl> impl_; // 头文件里只有一个指针
};

// widget.cpp
#include <heavy_library.h>
struct Widget::Impl { HeavyType data; void work(); };
Widget::Widget() : impl_(std::make_unique<Impl>()) {}
Widget::~Widget() = default; // ★ 此处 Impl 是完整类型
void Widget::do_work() { impl_>work(); }
```

好处 1. 头文件不 include 重依赖 → **全项目编译快很多** 2. 改实现不触发下游重编译 3. **ABI 稳定** (类大小永远是一个指针) —— 做动态库/SDK 必备

代价: 一次额外堆分配 + 一次间接访问。热点小对象别用。

【坑】析构函数必须在 .cpp 里定义 (哪怕是 `= default`)。否则在头文件处 `unique_ptr<Impl>` 析构时 `Impl` 还是不完整类型, 编译报错。

7.3 CRTP 与 类型擦除

CRTP（奇异递归模板模式）—— 静态多态

```
template <typename Derived>
class Printable {
public:
    void print() const {
        std::cout << static_cast<const Derived*>(*this).to_string();
    }
};

class Money : public Printable<Money> {
public:
    std::string to_string() const { return "..."; }
};
```

基类通过 `static_cast<Derived*>(*this)` 调到派生类，全部编译期解析，可内联，零虚函数开销。

【用在哪】 给一堆类批量加公共功能（mixin）、表达式模板、单例基类、统计对象数量。

注意：C++20 有了 `<=>` 和 `concepts` 后，CRTP 的一部分传统用途（比较运算符 mixin、接口约束）已经被取代了。

类型擦除 —— 值语义 + 多态 + 不需要继承

```
class Drawable {
    struct Concept {
        virtual ~Concept() = default;
        virtual void draw() const = 0;
        virtual std::unique_ptr<Concept> clone() const = 0;
    };
    template <typename T>
    struct Model final : Concept {
        T obj;
        explicit Model(T o) : obj(std::move(o)) {}
        void draw() const override { obj.draw(); }           // 鸭子类型
        std::unique_ptr<Concept> clone() const override {
            return std::make_unique<Model>(obj);
        }
    };
    std::unique_ptr<Concept> self_;
public:
    template <typename T>
    Drawable(T obj) : self_(std::make_unique<Model<T>>(std::move(obj))) {}
    Drawable(const Drawable& o) : self_(o.self_>clone()) {}
    void draw() const { self_>draw(); }
};

struct Cat { void draw() const; };           // ← 不继承任何东西
struct Dog { void draw() const; };

std::vector<Drawable> zoo{Cat{}, Dog{}};
auto copy = zoo;                           // ✅ 可以整体拷贝（虚继承做不到）
```

【用在哪】 `std::function` / `std::any` 就是这么实现的。插件系统、想要「多态 + 值语义容器」时。

7.4 创建型

单例（Meyers Singleton）

```
class Logger {  
public:  
    static Logger& instance() {  
        static Logger inst;           // C++11 起局部静态初始化是【线程安全】的  
        return inst;  
    }  
    Logger(const Logger&) = delete;  
    Logger& operator=(const Logger&) = delete;  
private:  
    Logger() = default;  
};
```

警告：单例是全局状态。它会让单元测试变难（没法替换成 mock）、隐藏依赖关系、有静态析构顺序问题。
能用依赖注入（把对象当参数传进去）就别用单例。

工厂（注册式）—— 加新类型不用改工厂代码

```
class ShapeFactory {
public:
    using Creator = std::function<std::unique_ptr<Shape>(double)>;
    static ShapeFactory& instance() { static ShapeFactory f; return f; }
    void register_type(std::string k, Creator c) { creators_[std::move(k)] = std::move(c); }
    std::unique_ptr<Shape> create(const std::string& k, double p) const {
        auto it = creators_.find(k);
        return it == creators_.end() ? nullptr : it->second(p);
    }
private:
    std::map<std::string, Creator> creators_;
};

// 自动注册：静态对象在 main 之前构造
template <typename T>
struct Registrar {
    explicit Registrar(std::string k) {
        ShapeFactory::instance().register_type(std::move(k),
        [](double p) { return std::make_unique<T>(p); });
    }
};

static Registrar<Circle> reg_circle{"circle"};
```

加一个新形状只需要写一行 `static Registrar<Triangle> reg{"triangle"};`，工厂本身一行都不用改——这就是开闭原则。

【用在哪】 插件、配置驱动的对象创建、反序列化、消息分发。

建造者

```
auto req = HttpRequest::Builder{}
    .url("https://api.example.com")
    .method("POST")
    .header("Content-Type", "application/json")
    .timeout(3000)
    .build();
```

【用在哪】 构造参数多、大部分可选、且构造后不可变的对象。

现代替代：参数少时 C++20 指定初始化器更轻量：

```
struct Options { int retries = 3; bool verbose = false; };
configure({.retries = 5});
```

7.5 结构型

模式	一句话	【用在哪】
适配器	包一层，让老接口符合新接口	对接遗留代码 / 第三方库。标准库的 <code>stack / queue</code> 就是 <code>deque</code> 的适配器
装饰器	层层包裹，动态添加职责	IO 流（缓冲/加密/压缩）、HTTP 中间件、UI 控件加边框滚动条
代理	替身，控制对真实对象的访问	延迟加载、远程代理(RPC stub)、权限检查、缓存。 智能指针就是所有权代理
外观	一个简单接口包住一堆子系统	SDK 对外 API、复杂库的便捷封装
桥接	抽象和实现两条继承线各自演化	避免 $N \times M$ 类爆炸。3 种形状 $\times$ 4 种渲染器：继承要 12 个类，桥接只要 7 个
组合	树形结构，叶子和容器一视同仁	文件系统、UI 控件树、AST、组织架构
享元	共享不变的内部状态	字体渲染、游戏里的树/草模型、字符串驻留、连接池

装饰器的调用链

```
auto src = std::make_unique<Encrypted>(
    std::make_unique<Compressed>(
        std::make_unique<FileSource>()));
src->read();
// 执行顺序: Encrypted::read -> Compressed::read -> FileSource::read
//          然后反向返回，每层做自己的解压/解密
```

7.6 策略 —— 三种实现方式，怎么选

```
// (a) 虚函数：运行期可切换，有间接调用开销
class Compressor { public: virtual std::string compress(std::string_view) = 0; };
archiver.set_policy(std::make_unique<GzipPolicy>());

// (b) std::function：最灵活，可以直接塞 lambda
std::function<std::string(std::string_view)> policy = [](auto s) { return ...; };

// (c) 模板策略：编译期确定，可内联，零开销
template <typename Policy>
class FastArchiver {
public:
    auto archive(std::string_view s) { return Policy::compress(s); }
};
FastArchiver<Gzip> a;
```

场景	选哪个
运行期要换（用户配置、插件）	虚函数 或 <code>std::function</code>
编译期就定死，且在热点路径	模板策略
只是一个小回调	直接传 lambda，别搞类

7.7 观察者（信号槽）

```
template <typename... Args>
class Signal {
public:
    using Slot = std::function<void(Args...)>;
    using Token = std::size_t;
    Token connect(Slot s) { slots_.emplace(next_, std::move(s)); return next_++; }
    void disconnect(Token t) { slots_.erase(t); }
    void emit(Args... args) const { for (auto& [id, s] : slots_) s(args...); }
private:
    std::map<Token, Slot> slots_;
    Token next_ = 0;
};

class Button { public: Signal<int, int> on_click; };
auto tok = btn.on_click.connect([](int x, int y) { ... });
btn.on_click.disconnect(tok);
```

【坑】生命周期 —— 观察者先于被观察者析构 = 悬垂回调 = 崩溃。

两种解法：1. connect 返回 token，观察者析构时 disconnect（上面这样） 2. 观察者列表存 weak_ptr，emit 时 lock()，失效的顺手清掉：

```
void notify(const Event& e) {
    std::erase_if(observers_, [&](const std::weak_ptr<IObserver>& w) {
        if (auto s = w.lock()) { s->on_event(e); return false; }
        return true; // 已死，移除
    });
}
```

【用在哪】 GUI 事件、模型-视图同步、发布订阅、状态变更通知、插件钩子。

7.8 命令 + 撤销/重做

```
class Command {
public:
    virtual ~Command() = default;
    virtual void execute() = 0;
    virtual void undo() = 0;
};

class History {
    std::vector<std::unique_ptr<Command>> done_, undone_;
public:
    void run(std::unique_ptr<Command> c) {
        c->execute();
        done_.push_back(std::move(c));
        undone_.clear();           // 新操作清空重做栈
    }
    void undo() {
        if (done_.empty()) return;
        done_.back()->undo();
        undone_.push_back(std::move(done_.back()));
        done_.pop_back();
    }
    void redo() { /* 反过来 */ }
};
```

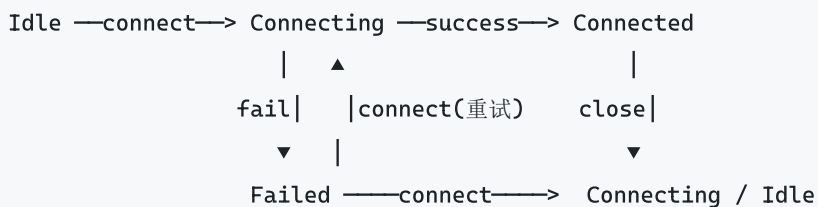
要点：命令对象要自带「怎么撤销」所需的全部信息（比如 `InsertText` 要记住插入位置和内容，才能撤销时精确删除）。

【用在哪】 编辑器撤销栈、事务、任务队列、宏录制、请求重放、游戏回放。

7.9 状态机 (variant 版) —— 强烈推荐

```
struct Idle {};  
struct Connecting { int attempts; };  
struct Connected { int fd; };  
struct Failed { std::string reason; };  
using ConnState = std::variant<Idle, Connecting, Connected, Failed>;  
  
ConnState on_event(const ConnState& s, std::string_view ev) {  
    if (ev == "connect") {  
        return std::visit(overloaded{  
            [](Idle) -> ConnState { return Connecting{1}; },  
            [](const Connecting& c) -> ConnState {  
                return c.attempts >= 3 ? ConnState{Failed{"重试超限"}}  
                    : ConnState{Connecting{c.attempts + 1}};  
            },  
            [](const Connected& c) -> ConnState { return c; },  
            [](const Failed&) -> ConnState { return Connecting{1}; },  
        }, s);  
    }  
    // ...  
}
```

状态转移图



为什么比 `enum` + 一堆成员变量好

1. 每个状态带自己独有的数据 (Connecting 有 attempts, Connected 有 fd)
2. 不合法的状态根本表示不出来 (不会出现 "Idle 状态但 fd 有值" 这种脏数据)
3. 漏处理一个状态, `visit` 编译期就报错
4. 无堆分配, 全在栈上

7.10 访问者 (variant 版) 与「表达式问题」

```
using ShapeV = std::variant<Circle, Square, Triangle>;

double area(const ShapeV& s) {
    return std::visit(overloaded{
        [](const Circle& c)    { return 3.14159 * c.r * c.r; },
        [](const Square& q)    { return q.s * q.s; },
        [](const Triangle& t) { return 0.5 * t.b * t.h; },
    }, s);
}
```

这是个经典权衡 ("表达式问题")

	加新类型	加新操作
虚函数继承	✔ 容易 (新写一个派生类)	✘ 难 (要改所有类)
variant + visit	✘ 难 (要改所有 visit)	✔ 容易 (新写一个函数)

怎么选： - 类型集合**固定** (AST 节点、协议消息、状态、token) → **variant** - 类型集合会**不断增加** (插件、图形对象、设备驱动) → **虚函数**

7.11 责任链（中间件管道）

```
using Next      = std::function<void()>;
using Middleware = std::function<void(Request&, Next)>;

class Pipeline {
    std::vector<Middleware> mws_;
public:
    Pipeline& use(Middleware m) { mws_.push_back(std::move(m)); return *this; }
    void run(Request& r) { invoke(r, 0); }
private:
    void invoke(Request& r, size_t i) {
        if (i >= mws_.size()) return;
        mws_[i](r, [this, &r, i] { invoke(r, i + 1); });
    }
};

pipe.use([](Request& r, Next next) {
    log_start(r);
    next();                // 调下一层
    log_end(r);            // 下一层返回后还能做事（洋葱模型）
})
.use([](Request& r, Next next) {
    if (!authed(r)) return; // 不调 next(), 链在这里断开
    next();
})
.use([](Request& r, Next) { handle(r); });
```

洋葱模型：请求一层层进去，响应一层层出来。

```
请求  →  [日志前] → [鉴权] → [处理器]
                                   |
响应  ←  [日志后] ←—————┘
```

【用在哪】 Web 框架中间件、拦截器、日志/鉴权/限流/压缩层、事件处理链。第 12 章的 HTTP 服务器就是这么组织的。

7.12 模式选择速查

需求	用什么
管资源	RAII (永远先想这个)
运行期换算法	策略 (<code>std::function</code> / 虚函数)
编译期定死算法且在热点路径	模板策略
一对多通知	观察者 / 信号槽
撤销重做 / 请求排队	命令
隐藏实现、降低编译依赖、稳定 ABI	Pimpl
类型集合封闭 + 多种操作	<code>variant</code> + <code>visit</code>
类型集合开放 + 操作固定	虚函数继承
要值语义又要多态	类型擦除
两个维度各自变化 (N×M)	桥接
树形结构统一处理	组合
动态叠加职责	装饰器
请求可能被多个处理者处理	责任链 / 中间件
大量重复的不可变对象	享元

7.13 反模式提醒

不要为了用模式而用模式。 现代 C++ 里很多 GoF 模式已经被语言吃掉了：

传统模式	现代替代
简单 Strategy / Command	一个 lambda
Visitor	<code>std::variant</code> + <code>std::visit</code>
比较运算符 mixin (CRTP)	<code>auto operator<==>() = default</code>
简单 Builder	C++20 指定初始化器
Iterator	直接满足 ranges 概念
Prototype (clone)	拷贝构造 + 值语义
Singleton	依赖注入 (大多数情况)

模式是用来沟通的词汇，不是必须完成的仪式。 代码里出现 `AbstractFactoryProxyBuilderImpl` 这种名字，说明方向错了。

第 8 章 · 网络基础：什么是网络

► 对应程序： `ch08_net_basics`

8.1 从最基本的问题开始

问题： A 电脑上的一个程序，怎么把一段数据交给 B 电脑上的另一个程序？

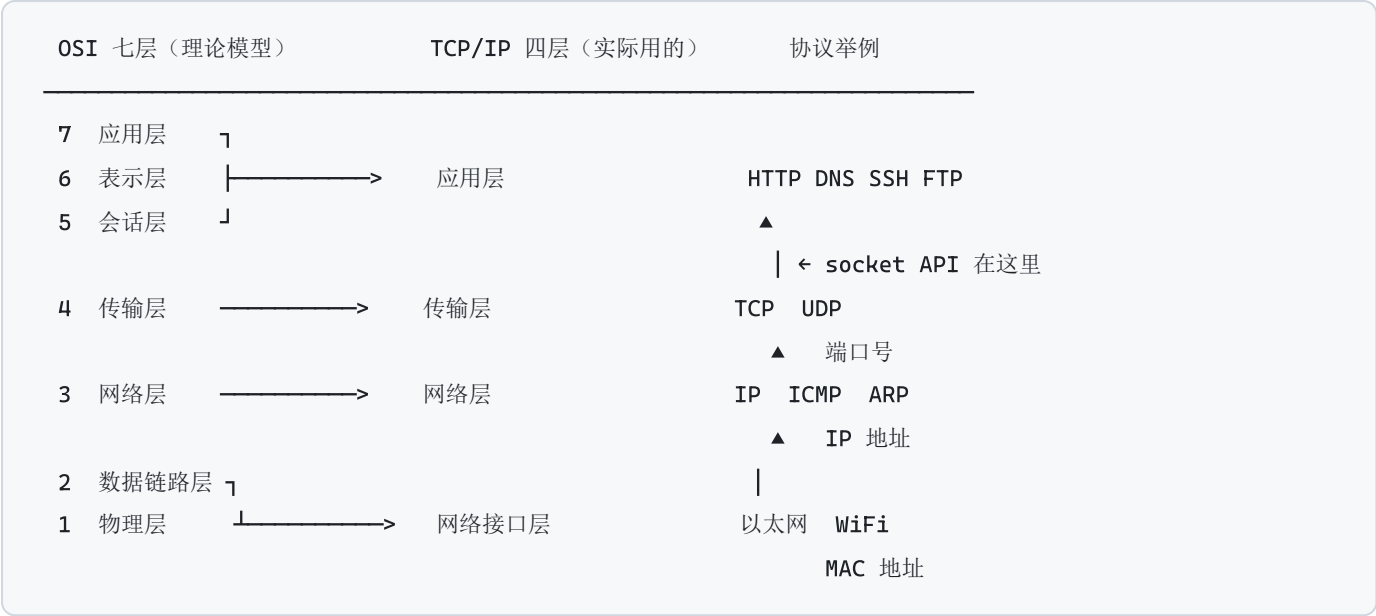
需要解决四件事：

问题	解决方案
找到 B 这台机器	IP 地址
找到 B 上的那个程序	端口号
数据怎么在物理线路上跑	以太网 / WiFi / 光纤 + 逐层封装
丢了怎么办、乱序怎么办	TCP（或者不管，那就是 UDP）

「网络协议」就是通信双方约定好的一套规则，规定了：数据长什么样（格式）、什么时候发什么（时序）、出错怎么处理（差错控制）。

8.2 分层模型 —— 为什么要分层

分层的核心思想：**每一层只管自己的事，把下层当成能用的黑盒。** 这样换网卡不用改浏览器，换 WiFi 不用改 TCP。



socket 编程发生在应用层和传输层的交界处：你调用 `send()`，内核负责 TCP 分段、加 IP 头、交给网卡驱动。

8.3 封装与解封装



接收端反过来一层层拆开（解封装），每层只看自己那个头。

关键概念：MTU 与 MSS

- MTU（最大传输单元）：链路层一帧最多能装多少字节。以太网通常是 1500。
- 超过 MTU 就要 IP 分片。分片很糟：任何一片丢了整个包都要重传，效率骤降。
- MSS（最大报文段长度）：TCP 一段最多装多少应用数据。 $MSS = MTU - IP头 - TCP头 = 1500 - 20 - 20 = 1460$ （典型值）

- TCP 会在握手时协商 MSS，主动按 MSS 切分，**避免 IP 分片**。
- UDP 不管这些，所以 **UDP 负载建议控制在 1400 字节以内**。

8.4 IP 地址与端口

IP 地址

- **IPv4**: 32 位，写成 4 段十进制，如 192.168.1.100
- **IPv6**: 128 位，写成 8 组十六进制，如 2001:db8::1

地址	含义
127.0.0.1	回环地址，永远指向本机，数据 不出网卡
0.0.0.0	服务端 bind 时表示「监听所有网卡」
192.168.x.x / 10.x.x.x / 172.16~31.x.x	私有地址，只在局域网有效，需 NAT 才能上外网
255.255.255.255	广播地址
224.0.0.0 ~ 239.255.255.255	组播地址

端口

16 位无符号数，0~65535。

范围	名称	说明
0~1023	知名端口	需要 root/管理员权限才能绑定
1024~49151	注册端口	应用程序申请注册的
49152~65535	动态/临时端口	客户端连接时内核自动分配

常见知名端口：20/21 FTP、22 SSH、23 Telnet、25 SMTP、53 DNS、80 HTTP、443 HTTPS、3306 MySQL、6379 Redis、27017 MongoDB。

【面试】四元组

一条 TCP 连接由四元组唯一标识：（源IP，源端口，目的IP，目的端口）

所以一个服务端在 80 端口能同时服务几万个客户端 —— 它们的源 IP/源端口不同。理论上，单个服务端口能承载的连接数 = 客户端 IP 数 × 每个客户端的可用端口数，远超 65535 这个常见误解。

8.5 字节序（大端 / 小端）—— 网络编程第一个坑

同样一个 32 位整数 `0x12345678`，在内存里怎么摆？

大端 Big Endian

地址：低 —————> 高

字节： 12 34 56 78

高位字节放低地址（符合人阅读顺序）

小端 Little Endian

地址：低 —————> 高

字节： 78 56 34 12

低位字节放低地址

x86 / ARM 都是【小端】

网络字节序统一规定为【大端】（也叫 Network Byte Order）

`ch08_net_basics` 用 `hexdump` 实测能看到这个差异。

四个转换函数（必须背下来）

`htons(x)`

`// host to network short`

`— 16 位, 用于【端口】`

`htonl(x)`

`// host to network long`

`— 32 位, 用于【IPv4 地址】`

`ntohs(x)`

`// network to host short`

`ntohl(x)`

`// network to host long`

什么时候必须转换

场景	要转吗
填 <code>sockaddr_in</code> 的 <code>sin_port</code> / <code>sin_addr</code>	✅ 必须
自定义二进制协议里的多字节整数（长度前缀、ID 等）	✅ 必须（约定用网络序）
单个 <code>char</code> / 字符串 / 字节数组	❌ 不用（没有字节序问题）
文本协议（HTTP、JSON）	❌ 不用

【坑】忘记转换的典型症状：本机测试完全正常（两边都是小端，错得一致），一旦和别的架构机器或标准协议通信就全乱 —— 非常难查。

C++20 起可以用 `std::endian::native` 判断，C++23 有 `std::byteswap` 做翻转。

8.6 地址结构 `sockaddr` —— 一个历史包袱

socket API 是 1983 年 BSD 定下来的，那时候 C 语言还没有 `void*`，所以设计了一个「通用地址」`struct sockaddr`，各协议族再定自己的结构，传参时强制转换成 `sockaddr*` —— 这就是你到处看到 `(sockaddr*)`

的原因。

```
struct sockaddr {                // 通用, 16 字节, 只用来做参数类型
    sa_family_t sa_family;        // AF_INET / AF_INET6 / AF_UNIX
    char        sa_data[14];
};

struct sockaddr_in {             // IPv4 专用, 实际用这个
    sa_family_t  sin_family;      // AF_INET
    uint16_t     sin_port;        // 端口, 【网络字节序】
    struct in_addr sin_addr;      // IPv4 地址, 【网络字节序】
    char        sin_zero[8];     // 填充, 保证和 sockaddr 一样大
};

struct sockaddr_in6 { ... };     // IPv6, 28 字节
struct sockaddr_storage { ... }; // 足够大, 能装下任意协议族的地址
```

构造地址 (现代写法)

```
sockaddr_in a{};
a.sin_family = AF_INET;
a.sin_port   = htons(8080);      // ★ 端口要转
inet_pton(AF_INET, "192.168.1.100", &a.sin_addr); // 字符串 -> 二进制
// 或监听所有网卡:
a.sin_addr.s_addr = htonl(INADDR_ANY);

// 二进制 -> 字符串
char buf[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &a.sin_addr, buf, sizeof(buf));
```

别用 `inet_addr` / `inet_ntoa`：不支持 IPv6，且 `inet_ntoa` 用静态缓冲区（线程不安全）。

`ch08_net_basics` 会把 `sockaddr_in` 的原始字节打出来，能直接看到 `02 00`（AF_INET）、`1F 90`（8080 的网络序）、`C0 A8 01 64`（192.168.1.100）。

8.7 DNS 解析

```
addrinfo hints{};
hints.ai_family   = AF_UNSPEC;      // IPv4 和 IPv6 都要
hints.ai_socktype = SOCK_STREAM;    // TCP

addrinfo* res = nullptr;
if (getaddrinfo("example.com", "443", &hints, &res) == 0) {
    for (addrinfo* p = res; p; p = p->ai_next) {
        // 依次尝试连接每个地址
    }
    freeaddrinfo(res);              // ★ 必须释放
}
```

要点

- 1. 返回的是**链表**，一个域名可能有多个 IP，应该**依次尝试**
- 2. 结果必须 `freeaddrinfo` 释放
- 3. 它是**阻塞**调用，慢的 DNS 能卡好几秒 —— 高性能服务要用异步 DNS 库（c-ares）
- 4. 老 API `gethostbyname` 不支持 IPv6 且线程不安全，别用
- 5. 它还能把服务名翻译成端口： `getaddrinfo(host, "http", ...)` → 80

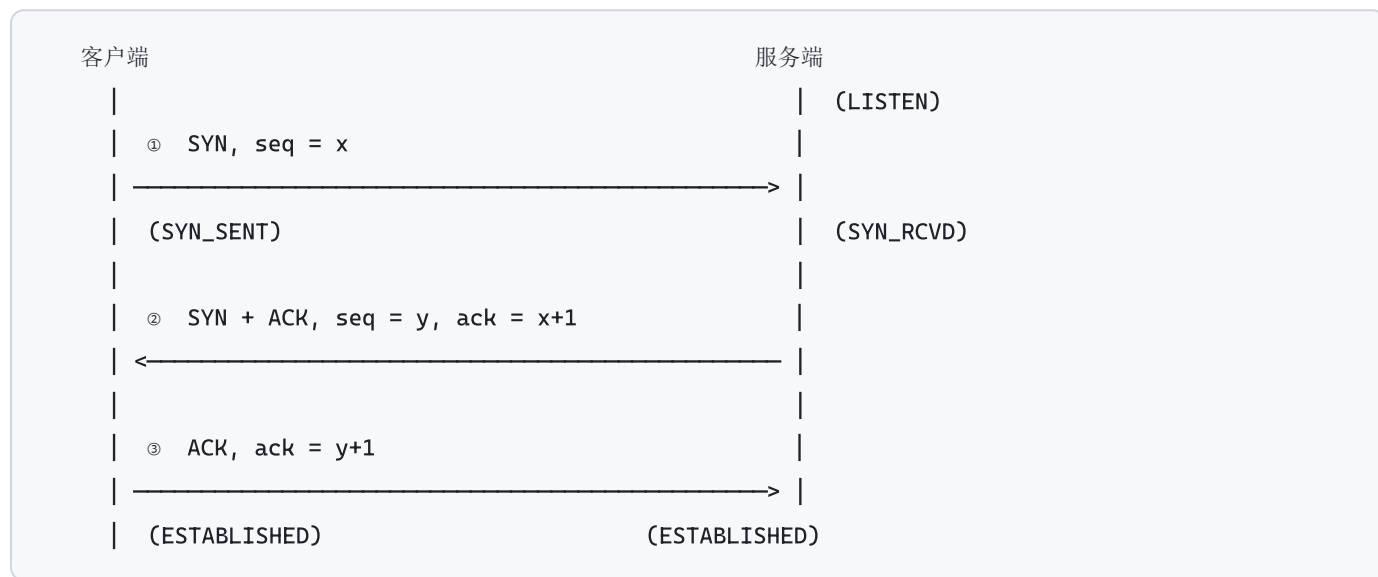
8.8 TCP vs UDP

	TCP	UDP
连接	面向连接（先三次握手）	无连接（直接发）
可靠性	保证到达、不重复	不保证，可能丢/重复
顺序	保证按发送顺序交付	不保证
流量控制	有（滑动窗口）	无
拥塞控制	有（慢启动/拥塞避免/快重传）	无
数据边界	无！是字节流	有！一个包就是一个消息
头部大小	20 字节起	8 字节
速度	慢一些	快
一对多	不支持	支持广播/组播
典型场景	HTTP/HTTPS、SSH、数据库、文件传输、邮件	DNS、直播、游戏、VoIP、SNMP、QUIC 的底座

选型口诀

「数据不能错、不能少、顺序不能乱」→ **TCP** 「宁可丢也不能卡，或者要一对多」→ **UDP** (应用层自己做必要的可靠性)

8.9 TCP 三次握手



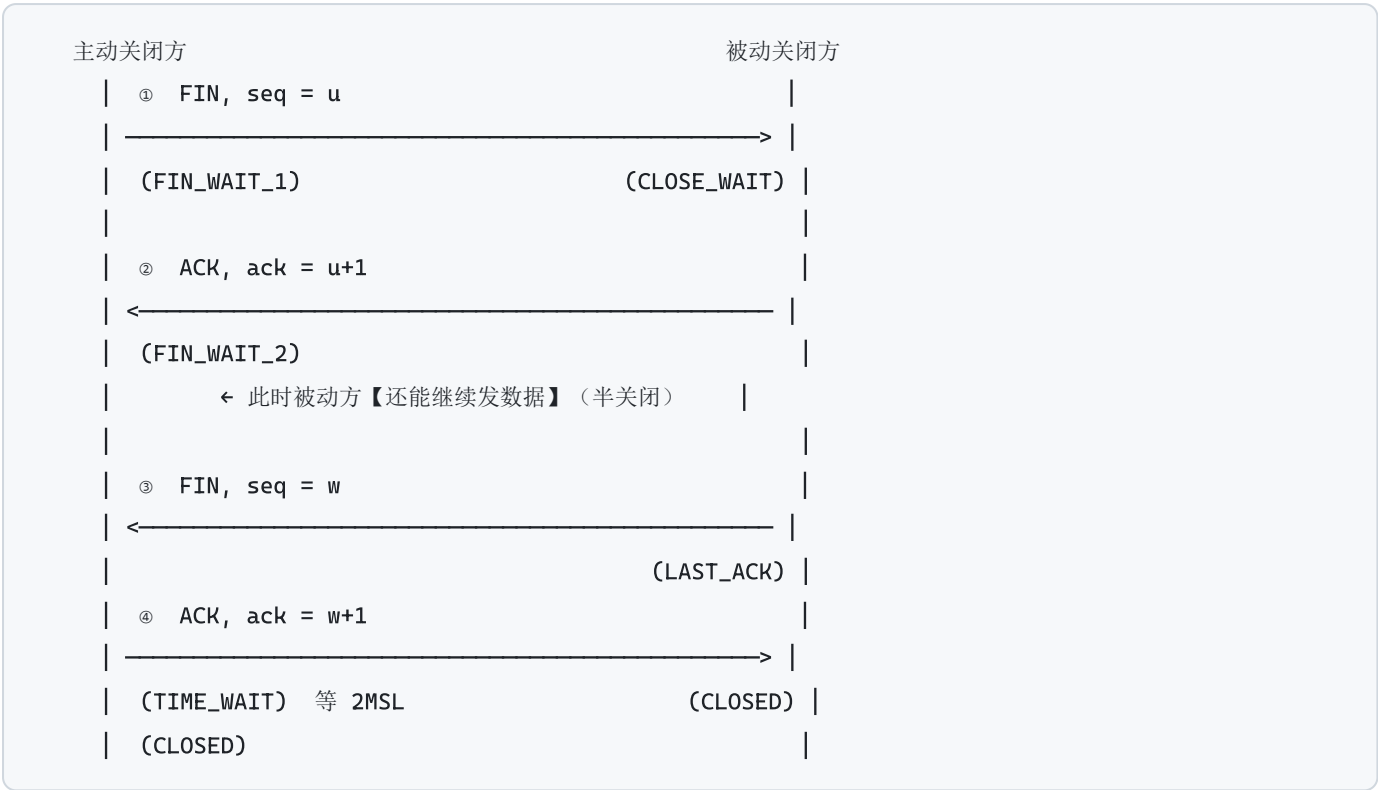
【面试】为什么是三次不是两次？

两次握手的话，服务端**无法确认「客户端真的收到了我的响应」**。考虑这个场景：客户端第一个 SYN 因网络拥堵延迟了很久，客户端超时重发了新 SYN 并完成通信、关闭连接。此时那个**旧的 SYN 才姗姗来迟**到达服务端——两次握手下服务端会直接建立连接并一直占着资源，而客户端根本不认这个连接。

第三次 ACK 让**双方都确认了对方的收发能力都正常**。

【面试】为什么不是四次？ 第二步的 SYN 和 ACK 可以合并成一个包发，没必要拆开。

8.10 TCP 四次挥手与 TIME_WAIT



【面试】为什么挥手要四次？

因为 TCP 是**全双工**的，两个方向要**分别关闭**。被动方收到 FIN 后可能还有数据没发完，所以 ACK 和自己的 FIN 要分开发。（如果没数据要发，有些实现会合并成三次。）

【面试】TIME_WAIT 为什么要等 2MSL（Linux 上通常 60 秒）？

MSL = Maximum Segment Lifetime，报文在网络中的最大存活时间。

- 1. **确保最后那个 ACK 能到达对方。**如果 ACK 丢了，对方会重发 FIN，自己还在 TIME_WAIT 就能再 ACK 一次。如果直接 CLOSED，就会回一个 RST，对方报错。
- 2. **让本连接的所有残留报文在网络中自然消亡，**避免污染下一个使用相同四元组的新连接。

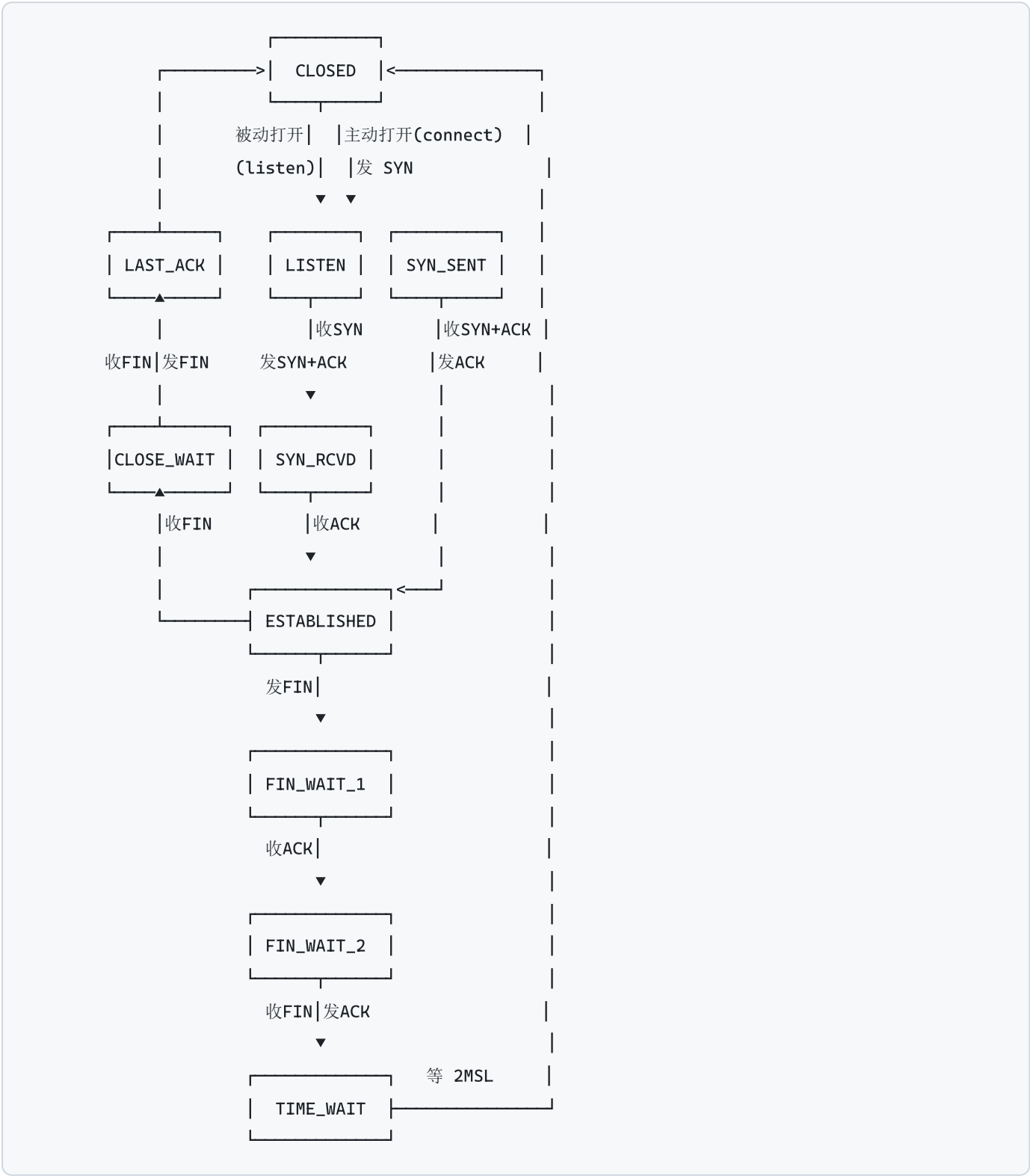
实际影响与缓解

服务端如果主动关连接（比如 HTTP 短连接），会积累大量 TIME_WAIT，占满本地端口，表现为「无法建立新连接」。

缓解手段	说明
SO_REUSEADDR	让重启的服务能立刻 bind 处于 TIME_WAIT 的端口（ 服务端必设 ）
用长连接（keep-alive）	从根本上减少连接建立/关闭次数 —— 最有效
让客户端主动关	TIME_WAIT 落在客户端，客户端数量多，压力分散
net.ipv4.tcp_tw_reuse=1	Linux 内核参数，允许复用 TIME_WAIT 连接（客户端侧安全）

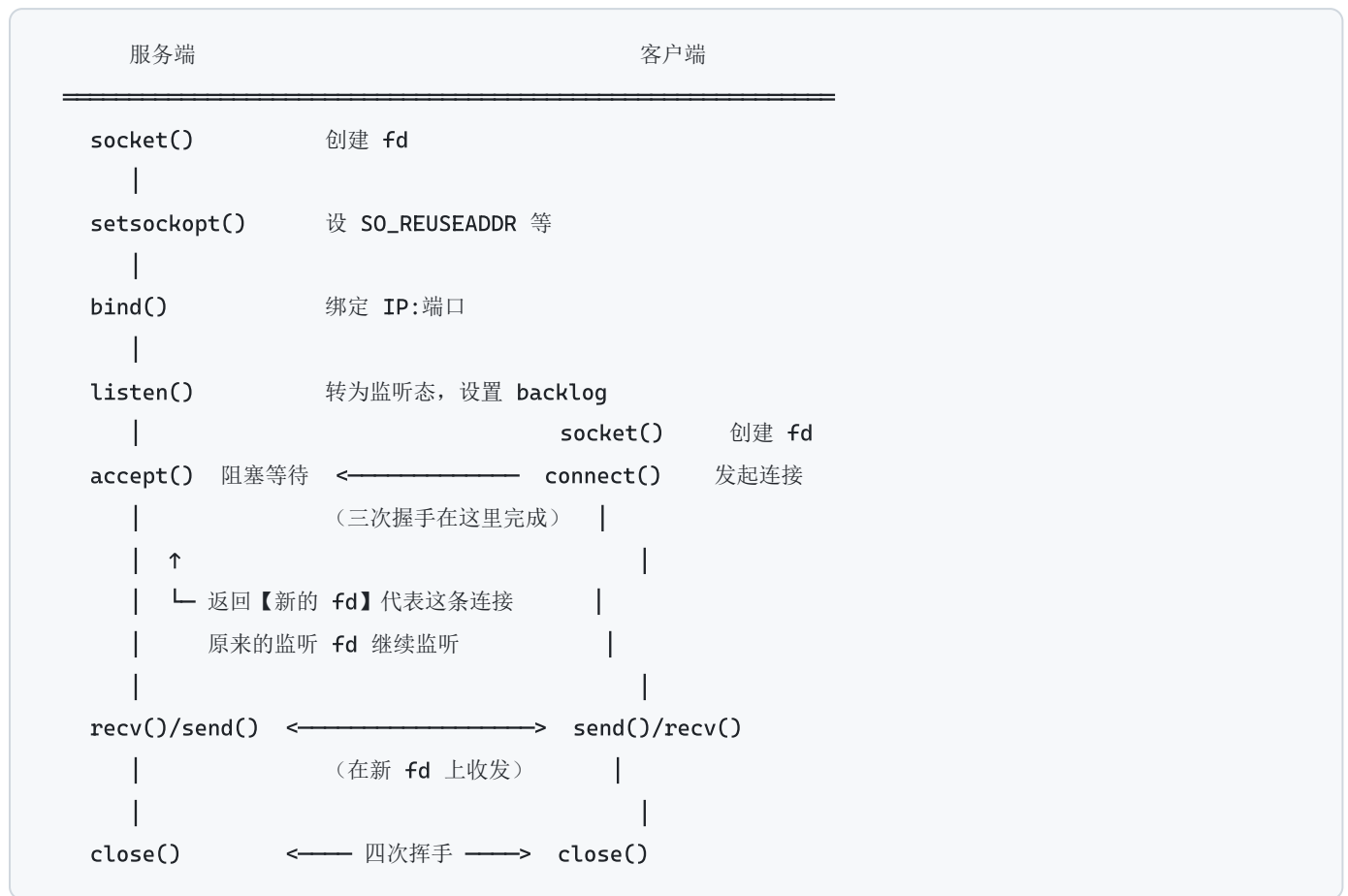
注意： `tcp_tw_recycle` 在 NAT 环境下会导致连接失败，Linux 4.12 起已被移除，网上很多老文章还在推荐它，**别用**。

8.11 TCP 状态机



用 `ss -tanp` (Linux) 或 `netstat -ano` (Windows) 能实时看到这些状态。

8.12 socket API 全景



每个调用干了什么

`socket(AF_INET, SOCK_STREAM, 0)` 创建一个 socket，返回文件描述符。Linux 上「一切皆文件」，所以 socket fd 可以用 `read / write / close`，也能放进 `select / epoll`。Windows 上不行，只能用 `recv / send`。

`bind(fd, addr, len)` 把 fd 和一个本地 IP:端口关联。客户端一般不用调（内核自动分配临时端口）。

`listen(fd, backlog)` 把 fd 从「主动」变成「被动监听」。

`backlog` 是已完成握手但还没被 `accept` 取走的连接队列长度。（内核里实际有两个队列：半连接队列 SYN queue 和全连接队列 accept queue，backlog 影响后者。）队列满了新连接会被丢弃 → 客户端表现为连接超时。

`accept(fd, &addr, &len)` 从队列取一个已建立的连接，返回新 fd。阻塞模式下没连接就一直等。

`connect(fd, addr, len)` 客户端发起三次握手。阻塞模式下要等握手完成或超时（默认可能长达 75 秒以上）。

`send / recv`（或 `write / read`）

```
long n = recv(fd, buf, len, 0);
if (n > 0) { /* 收到 n 字节 */ }
if (n == 0) { /* ★ 对端【正常关闭】了连接，不是错误！ */ }
if (n < 0) { /* 出错。但要区分 EINTR（被信号打断，应重试）和
                EAGAIN/EWOULDBLOCK（非阻塞模式下暂时没数据，不是错误） */ }
```

★ `send` 也可能只发出去一部分，必须循环：

```
bool send_all(int fd, const char* p, size_t len) {
    size_t sent = 0;
    while (sent < len) {
        long n = send(fd, p + sent, len - sent, 0);
        if (n > 0) { sent += n; continue; }
        if (n < 0 && errno == EINTR) continue;
        return false;
    }
    return true;
}
```

这是新手最常见的 bug 之一。

`close(fd)` 引用计数减一，到 0 才真正发 FIN。`shutdown(fd, SHUT_WR)` 可以**只关写方向**（半关闭），用来告诉对端「我发完了，但我还能收」—— HTTP/1.0 和某些协议会用到。

8.13 常用 socket 选项

选项	作用	什么时候设
SO_REUSEADDR	允许 bind 处于 TIME_WAIT 的端口	服务端几乎必设，否则重启报 "Address already in use"
SO_REUSEPORT	多个进程绑同一端口，内核做负载均衡 (Linux 3.9+)	多进程 accept，避免惊群
TCP_NODELAY	关闭 Nagle 算法	交互式协议（游戏/RPC/SSH）—— 见下文
SO_KEEPALIVE	开启 TCP 保活探测	检测对端「假死」。默认 2 小时太长，一般应用层自己做心跳
SO_RCVBUF / SO_SNDBUF	收发缓冲区大小	高带宽长延迟链路（跨国）需要调大，否则窗口不够跑不满带宽
SO_RCVTIMEO / SO_SNDTIMEO	收发超时	阻塞模式下防止永久卡死的简易办法
SO_LINGER	控制 close 时如何处理未发送数据	慎用，容易造成数据丢失

Nagle 算法：把小包攒一攒再发（等到收到上一个包的 ACK，或攒够一个 MSS）。目的是减少「1 字节数据 + 40 字节头」这种极低效的包。代价是**增加延迟**。交互式应用（每次就发几十字节，要求立即到达）应该 TCP_NODELAY 关掉它。

【坑】Nagle + 延迟 ACK 的经典组合问题：发送方等 ACK 才发下一个小包，接收方的延迟 ACK 又要等 40ms 才回 —— 造成 40ms 的莫名延迟。

8.14 阻塞与非阻塞，五种 IO 模型

阻塞模式（默认） recv() 没数据 → 线程挂起，直到有数据或出错。优点：代码线性好读。缺点：一个线程只能盯一个连接。

非阻塞模式 recv() 没数据 → 立刻返回 -1，errno = EAGAIN / EWOULDBLOCK。这不是错误，只是「现在没有」，必须专门判断。单纯轮询非阻塞 socket 会烧 CPU，所以要配合 IO 多路复用。

```
// Linux
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);

// Windows
u_long mode = 1;
ioctlsocket(fd, FIONBIO, &mode);
```

五种 IO 模型（《UNIX 网络编程》经典分类）

1) 阻塞 IO

应用 —recv()—> 内核 [等数据][拷贝数据] —> 返回
←———— 全程阻塞 —————>

2) 非阻塞 IO

应用 —recv()—> EAGAIN （轮询，烧 CPU）
—recv()—> EAGAIN
—recv()—> 内核 [拷贝数据] —> 返回

3) IO 多路复用

← 主流

应用 —select/epoll()—> 内核 [等任一 fd 就绪] —> 返回就绪列表
—recv()————> 内核 [拷贝数据] —> 返回
一个线程能盯住成千上万个 fd

4) 信号驱动 IO

内核数据就绪时发 SIGIO 信号通知。实际很少用。

5) 异步 IO (AIO)

应用 —aio_read()—> 立刻返回，继续干别的
←—通知———— 内核 [等数据][拷贝数据] 全干完了才通知
Linux io_uring / Windows IOCP

【面试】前 4 种都是「同步 IO」 —— 数据从内核缓冲区拷贝到用户缓冲区 这一步是**你自己在等**。只有第 5 种是内核帮你拷完再通知你。

8.15 在 Windows 上学 Linux 网络编程

本教程的网络代码按 POSIX 语义写，通过 `src/common/net_compat.h` 兼容 Windows。

项目	Linux (POSIX)	Windows (Winsock2)
头文件	<sys/socket.h> 等	<winsock2.h> <ws2tcpip.h>
初始化	不需要	WSAStartup / WSACleanup
句柄类型	int (就是 fd)	SOCKET (UINT_PTR)
无效值	-1	INVALID_SOCKET
关闭	close()	closesocket()
错误码	errno	WSAGetLastError()
非阻塞	fcntl(O_NONBLOCK)	ioctlsocket(FIONBIO)
多路复用	select / poll / epoll	select / WSAPoll / IOCP
一切皆文件	✅ 可用 read / write	❌ 只能 recv / send

装 WSL2 跑真正的 Linux (学 epoll 必须) :

```
wsl --install           # 管理员 PowerShell, 然后重启

sudo apt update && sudo apt install -y build-essential gdb cmake
cd /mnt/c/Users/coder/Desktop/Claude_Code/CppLearning
cmake -B build-linux && cmake --build build-linux -j
./build-linux/bin/ch11_multiplex 8890 epoll
```

8.16 排障工具速查

连通性

```
ping <host>           # ICMP, 可能被防火墙挡, 不通不代表服务不可用
telnet <host> <port>   # 测某个 TCP 端口通不通 -- 最简单有效
nc -zv <host> <port>   # 同上, Linux 常用
curl -v http://host:port/ # 看完整 HTTP 交互
traceroute <host>      # 看路径上每一跳 (Windows 是 tracert)
```

看本机连接

```
# Linux
ss -tlnp # 所有监听端口 + 进程（推荐，比 netstat 快）
ss -tanp # 所有 TCP 连接及状态
ss -s # 汇总统计（各状态连接数）
lsof -i :8080 # 谁占了 8080
netstat -tlnp # 老版本

# Windows
netstat -ano | findstr :8080
Get-NetTCPConnection -LocalPort 8080
```

抓包

```
tcpdump -i any -nn port 8080 -w capture.pcap # 抓包存文件
tcpdump -i any -nn -A port 8080 # 直接看 ASCII 内容
wireshark capture.pcap # 图形化分析
```

Wireshark 里看三次握手、重传、RST、窗口变化最直观，过滤器写 `tcp.port == 8080` 或 `tcp.flags.syn == 1`。

看程序行为

```
strace -e trace=network ./prog # 看程序调了哪些网络系统调用
ltrace ./prog # 看库函数调用
```

常见现象与原因对照

现象	原因
Connection refused	目标端口 没人监听 （内核直接回 RST）
Connection timed out	包被 丢弃 ，没有任何响应（防火墙 DROP / 路由不通 / 对方过载）
Address already in use	端口被占，或处于 TIME_WAIT 且没设 SO_REUSEADDR
Broken pipe / EPIPE	往一个对端已关闭的连接上写（默认还会收到 SIGPIPE 信号，要忽略它）
Connection reset by peer	对端发了 RST ：进程崩了 / 强制关闭 / 收到非法包 / backlog 满
Too many open files	fd 耗尽。ulimit -n 调大，并检查是否有 fd 泄漏
连接偶发很慢（40ms 左右）	Nagle + 延迟 ACK 的经典组合，设 TCP_NODELAY
大量 TIME_WAIT	服务端主动关连接。改长连接，或调内核参数
大量 CLOSE_WAIT	你的程序收到 FIN 后没有 close() —— 这是代码 bug，赶紧查

`CLOSE_WAIT` 堆积几乎总是应用层 bug，值得单独记住：对端关了，你的 `recv` 返回 0，但你没调 `close()`。

第 9 章 · TCP 编程实战

► 对应程序: `ch09_tcp_server` + `ch09_tcp_client`

```
# 终端 1
build\bin\ch09_tcp_server.exe 8888 thread

# 终端 2
build\bin\ch09_tcp_client.exe 127.0.0.1 8888          # 交互模式
build\bin\ch09_tcp_client.exe 127.0.0.1 8888 bench    # 粘包演示 + 压测
```

9.1 服务端的固定套路

```
// ① 创建
socket_t s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

// ② 设选项 (bind 之前)
int on = 1;
setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));

// ③ 绑定
sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = htonl(INADDR_ANY);
bind(s, (sockaddr*)&addr, sizeof(addr));

// ④ 监听
listen(s, 128);

// ⑤ 循环 accept
for (;;) {
    sockaddr_in peer{};
    socklen_t len = sizeof(peer);
    socket_t c = accept(s, (sockaddr*)&peer, &len);
    handle(c);    // ★ c 是【新的】fd, s 继续监听
}
```

这五步的顺序不能变，`setsockopt(SO_REUSEADDR)` 必须在 `bind` 之前。

9.2 三种并发模型

`ch09_tcp_server` 用第二个参数切换，可以实际对比。

模型一： `iterative` 单线程串行

```
while (true) {  
    auto conn = accept(listener);  
    handle_connection(conn);    // 处理完才回来 accept 下一个  
}
```

-  最简单，没有任何并发问题
-  一次只能服务一个客户端，第二个要一直等
- **【用在哪】** 只有内部工具、调试用途

模型二： `thread` 每连接一个线程

```
while (true) {  
    auto conn = accept(listener);  
    std::thread([c = std::move(conn)] { handle_connection(c); }).detach();  
}
```

-  写起来简单，每个连接一个线性流程，容易理解和调试
-  线程栈默认 1~8 MB → **1 万连接要 10~80 GB 内存**
-  线程切换开销随线程数暴涨
-  恶意客户端狂建连接就能打垮你

这就是著名的 **C10K 问题**（如何让单机处理 1 万并发连接）。

【用在哪】 连接数少（几十到几百）、每个连接计算量大的场景。比如内部 RPC 服务、管理后台。

模型三：pool 线程池

```
ThreadPool pool(std::thread::hardware_concurrency());
while (true) {
    auto conn = accept(listener);
    pool.submit([c = std::move(conn)] { handle_connection(c); });
}
```

-  线程数固定（通常 = CPU 核数或其小倍数），不会无限膨胀
-  如果某个连接长时间不发数据，会白占一个工作线程
- 100 个空闲长连接就能把 8 线程的池占满

【用在哪】短连接为主、请求处理快的场景。

彻底的解法

IO 多路复用 + 非阻塞（第 11 章）。让一个线程管上万个连接，只有真正有数据的连接才占用 CPU。

生产架构通常是**多 Reactor + 线程池**：N 个事件循环线程负责 IO，业务计算丢给独立线程池。

9.3 处理一条连接：三个必须记住的细节

```
void handle_connection(socket_t conn) {
    char buf[4096];
    for (;;) {
        long n = recv(conn, buf, sizeof(buf), 0);

        if (n == 0) {
            // ★① 对端【正常关闭】（发来了 FIN），不是错误
            break;
        }
        if (n < 0) {
            // ★② 出错，但要区分：
            if (errno == EINTR) continue;           // 被信号打断，重试
            if (errno == EAGAIN) break;             // 非阻塞下暂时没数据
            break;                                   // 真错误
        }

        // ★③ send 可能只发出去一部分，必须循环
        if (!send_all(conn, buf, n)) break;
    }
    close(conn);
}
```

9.4 【重点】TCP 粘包与拆包

这是 TCP 编程最核心的概念。

ch09_tcp_client bench 会实测：客户端连续三次 `send("AAA")` `send("BBB")` `send("CCC")`，服务端一次 `recv` 收到 `"AAABBBCCC"` 9 个字节。

发送端	网络	接收端
<code>send("AAA")</code>		
<code>send("BBB")</code>	└─> 内核缓冲区 -> [AAABBBCCC] -> <code>recv()</code> 一次全收到	← 粘包
<code>send("CCC")</code>		

或者

`send("很长的一条消息...")` -> 超过 MSS，被切成两段 -> `recv()` 只收到前半截 -> 拆包

为什么会这样：TCP 是字节流协议。它只保证：- 字节按序到达 - 不丢、不重

它**不保证**你的一次 `send` 对应对方的一次 `recv`。内核会根据 Nagle 算法、缓冲区状态、MSS、网络拥塞情况自行决定怎么打包。

注意：「粘包」这个词其实是中文社区的俗称，容易让人以为是 bug。实际上这是 TCP 的设计本意——它就是个字节流管道，没有「包」这个概念。

三种解决方案

方案 1：固定长度

```
[==== 64 字节 ====][==== 64 字节 ====]
```

简单，但浪费空间，且长度一旦定死很难改。适合定长的传感器数据。

方案 2：长度前缀 (TLV) —— 最常用

```
[4字节长度][消息体][4字节长度][消息体]
```

```
// 发送
uint32_t len = htonl(static_cast<uint32_t>(body.size())); // ★ 转网络字节序
send_all(fd, &len, 4);
send_all(fd, body.data(), body.size());

// 接收
uint32_t netlen;
recv_exactly(fd, &netlen, 4); // ① 先收满 4 字节
uint32_t len = ntohl(netlen);
if (len > kMaxMessageSize) { // ★★ 必须校验上限！
    close(fd); // 否则恶意客户端发 len=0xFFFFFFFF
    return; // 你一 resize 内存就爆了 —— DoS 漏洞
}
std::string body(len, '\0');
recv_exactly(fd, body.data(), len); // ② 再按长度收满
```

`recv_exactly` 的实现

```
bool recv_exactly(socket_t s, void* buf, size_t n) {
    char* p = static_cast<char*>(buf);
    size_t got = 0;
    while (got < n) {
        long r = recv(s, p + got, n - got, 0);
        if (r > 0) { got += r; continue; }
        if (r < 0 && errno == EINTR) continue;
        return false; // 对端关闭或出错
    }
    return true;
}
```

方案 3：分隔符

```
消息1\n消息2\n消息3\n
```

HTTP 头部用 `\r\n` 分隔行、`\r\n\r\n` 表示头部结束，就是这个方案。缺点：内容里出现分隔符要转义。

混合方案：HTTP 实际上是「分隔符找头部 + 长度前缀 (Content-Length) 读正文」，兼顾了可读性和二进制安全——第 12 章会实现。

9.5 客户端要点

```
// 现代做法: getaddrinfo 解析 + 依次尝试
addrinfo hints{};
hints.ai_family = AF_UNSPEC; // IPv4/IPv6 都接受
hints.ai_socktype = SOCK_STREAM;

addrinfo* res;
getaddrinfo(host, port, &hints, &res);
for (addrinfo* p = res; p; p = p->ai_next) {
    int s = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (s < 0) continue;
    if (connect(s, p->ai_addr, p->ai_addrlen) == 0) break; // 连上就用它
    close(s); // 连不上试下一个
}
freeaddrinfo(res);
```

客户端不用 bind —— 内核在 `connect` 时自动分配一个临时端口 (49152~65535)。 `ch09_tcp_client` 会用 `getsockname` 把这个端口打出来。

连接超时怎么做：阻塞 `connect` 的默认超时很长（可能 75 秒以上）。正确做法是设成非阻塞，`connect` 立刻返回 `EINPROGRESS`，然后用 `select / poll` 等待可写，带自己的超时时间。

9.6 实测数据（回环地址）

`ch09_tcp_client bench` 在本机跑 1000 次请求-响应往返：

完成 1000 次请求-响应往返

总耗时 33 ms

平均往返延迟（RTT）33 us

QPS 约 30000

参考量级

链路	典型 RTT
回环 127.0.0.1	20~50 μ s
同机房	0.1~1 ms
同城	1~5 ms
跨城（如北京-上海）	10~30 ms
跨国（如中美）	150~300 ms

这就是为什么「减少往返次数」比「压缩数据量」更能提升体感。 一个需要 5 次串行往返的协议，跨国场景下光是等待就要 1.5 秒。HTTP/2 的多路复用、HTTP/3 的 0-RTT 握手，本质上都在解决这个问题。

第 10 章 · UDP 编程

► 对应程序： `ch10_udp_server` + `ch10_udp_client`

```
build\bin\ch10_udp_server.exe 9999
build\bin\ch10_udp_client.exe 127.0.0.1 9999 demo
```

10.1 和 TCP 的编程差异

<pre>// TCP socket(AF_INET, SOCK_STREAM, 0); bind(...); listen(...); accept(...); connect(...); recv(fd, buf, len, 0); send(fd, buf, len, 0);</pre>	<pre>// UDP socket(AF_INET, SOCK_DGRAM, 0); bind(...); // 服务端仍要 bind // X 没有 // X 没有 // 可选（见下） recvfrom(fd, buf, len, 0, &peer, &plen); sendto(fd, buf, len, 0, &peer, plen);</pre>
---	---

核心差异：UDP 没有「连接」的概念，所以每次收发都要带对端地址。

10.2 UDP 有消息边界

`ch10_udp_client` demo 实测：三次 `sendto("AAA")("BBB")("CCC")`，服务端三次 `recvfrom` 分别收到 AAA、BBB、CCC。

TCP:	<code>send("AAA") send("BBB") send("CCC")</code>	<code>-> recv()</code> 得到 "AAABBBCCC"	← 粘包
UDP:	<code>sendto("AAA") sendto("BBB")</code>	<code>-> recvfrom()</code> 得到 "AAA"	← 一一对应
		<code>recvfrom()</code> 得到 "BBB"	

这是 UDP 相对 TCP 最重要的语义差异。不用自己处理消息边界。

【坑】如果 `recvfrom` 的缓冲区比数据报小，多出来的部分会被直接丢弃（不像 TCP 会留在缓冲区等下次读）。所以缓冲区要开够（65536 保险）。

10.3 UDP 必须自己处理的事

① 超时 —— 没有连接，对端不回你就会永远卡在 `recvfrom` 上。

```
// Linux
timeval tv{1, 0}; // 1 秒
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

// Windows
DWORD ms = 1000;
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, (char*)&ms, sizeof(ms));
```

② **丢包 / 乱序 / 重复** —— 需要可靠性就得自己实现：序号 + ACK + 超时重传 + 去重 + 排序 + 流控。

这基本就是在重新发明 TCP。除非你有特殊需求，否则用 TCP。

真的需要「可靠 UDP」时，用成熟方案：

方案	场景
KCP	游戏，牺牲 10~20% 带宽换 30~40% 延迟降低
QUIC / HTTP3	Web，0-RTT 握手 + 避免队头阻塞 + 连接迁移
RTP / RTCP	音视频，允许丢帧但要求低延迟

③ **数据报大小** —— 理论上限 65507 字节，但超过 MTU(1500) 就要 IP 分片，分片中任何一片丢了整个数据报都废掉（丢包率被放大）。**经验值：单个 UDP 负载控制在 1400 字节以内。**

10.4 UDP 也可以 connect()

```
connect(udp_fd, &server_addr, sizeof(server_addr));
send(udp_fd, data, len, 0); // 之后可以直接用 send/recv
```

它不发任何包，只是在内核里记住默认对端。好处：

1. 之后可以用 `send / recv`，不用每次带地址
2. **能收到 ICMP 错误**（比如 Port Unreachable）—— 不 connect 的话这些错误被丢弃
3. 内核少做一次地址查找，性能略好
4. 内核会过滤掉其它地址发来的包

10.5 UDP 的独门能力：广播与组播

```
// 广播（只在本网段）
int on = 1;
setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &on, sizeof(on));
sendto(fd, data, len, 0, &broadcast_addr, sizeof(...)); // 目标 255.255.255.255

// 组播（一对多推送）
ip_mreq mreq{};
inet_pton(AF_INET, "239.1.1.1", &mreq.imr_multiaddr);
mreq.imr_interface.s_addr = htonl(INADDR_ANY);
setsockopt(fd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof(mreq));
```

TCP 完全不支持这两个 —— 这是 UDP 无法被替代的场景（服务发现、局域网设备探测、行情推送、IPTV）。

第 11 章 · IO 多路复用

► 对应程序： `ch11_multiplex`

```
build\bin\ch11_multiplex.exe explain           # 只看原理讲解
build\bin\ch11_multiplex.exe 8890 select       # 聊天室服务器（所有平台）
build\bin\ch11_multiplex.exe 8890 poll
./build-linux/bin/ch11_multiplex 8890 epoll    # 仅 Linux

# 测试：开 2~3 个终端各跑 telnet 127.0.0.1 8890，随便打字会广播给其他人
```

11.1 为什么需要它

- 「一连接一线程」的问题（C10K）：
- 线程栈默认 1~8 MB → 1 万连接就是 10~80 GB
 - 线程切换要保存/恢复寄存器、可能刷 TLB → 开销随线程数暴涨
 - 大部分线程其实在「等数据」，什么活都没干

多路复用的思路：一个线程问内核「这 10000 个 fd 里，现在哪些有数据了？」 内核回答后，线程只处理那几个就绪的。**一个线程顶一万个线程用。**

11.2 三者对比

	select	poll	epoll
fd 上限	FD_SETSIZE（通常 1024）	无	无
数据结构	位图 fd_set	pollfd 数组	内核红黑树 + 就绪链表
每次调用开销	全量拷贝进内核	全量拷贝进内核	只在 epoll_ctl 时改一次
找就绪 fd	遍历全部 O(n)	遍历全部 O(n)	直接返回就绪表 O(1)
集合会被改写	✅ 每次要重设	❌ events/revents 分开	❌
触发模式	只有 LT	只有 LT	LT / ET 都支持
可移植性	全平台	POSIX + WSAPoll	仅 Linux
适合	连接少	连接中等	海量连接

其它平台的对应物

平台	技术	模型
BSD / macOS	kqueue	Reactor (和 epoll 类似)
Windows	IOCP	Proactor (完成通知, 不是就绪通知)
Linux 新方案	io_uring	Proactor, 减少系统调用, 比 epoll 更进一步
跨平台封装	libevent / libuv / Boost.Asio / muduo	生产项目一般直接用这些

11.3 三种写法

select

```

fd_set readfds;
FD_ZERO(&readfds);
FD_SET(listener, &readfds);
int maxfd = listener;
for (auto fd : conns) { FD_SET(fd, &readfds); maxfd = std::max(maxfd, fd); }

timeval tv{1, 0};
int ready = select(maxfd + 1, &readfds, nullptr, nullptr, &tv);
//          ^^^^^^^^^ Linux 要 maxfd+1, Windows 忽略这个参数

if (FD_ISSET(listener, &readfds)) { /* 新连接 */ }
for (auto fd : conns) {
    if (FD_ISSET(fd, &readfds)) { /* 可读 */ }    // ← O(n) 遍历
}
// ★ fd_set 被内核改写了, 下一轮必须全部重新 FD_SET

```

poll

```
std::vector<pollfd> fds;
fds.push_back({listener, POLLIN, 0});
for (auto fd : conns) fds.push_back({fd, POLLIN, 0});

int ready = poll(fds.data(), fds.size(), 1000);

for (auto& pfd : fds) {
    if (pfd.revents & POLLIN) { /* 可读 */ }
    if (pfd.revents & (POLLHUP | POLLERR | POLLNVAL)) { /* 断开或出错 */ }
}
// events(关心什么) 和 revents(发生了什么) 分开, 不用重设
// 但每次还是要把整个数组拷进内核, 还是 O(n) 遍历
```

epoll (Linux)

```
int ep = epoll_create1(0); // ① 内核里建事件表

epoll_event ev{};
ev.events = EPOLLIN;
ev.data.fd = listener;
epoll_ctl(ep, EPOLL_CTL_ADD, listener, &ev); // ② 只在变化时调用

std::vector<epoll_event> events(1024);
for (;;) {
    int n = epoll_wait(ep, events.data(), events.size(), 1000); // ③ 取就绪列表
    for (int i = 0; i < n; ++i) { // ★ 只遍历【就绪的】
        int fd = events[i].data.fd;
        uint32_t e = events[i].events;
        if (e & EPOLLIN) { /* 可读 */ }
        if (e & EPOLLOUT) { /* 可写 */ }
        if (e & (EPOLLHUP | POLLERR)) { /* 断开 */ }
    }
}
```

为什么 epoll 快

select/poll:

用户态 [10000 个 fd]

| 每次全量拷贝



内核态 [遍历 10000 个]

| 找出就绪的



返回，用户再遍历 10000 个找就绪的

epoll:

用户态 (什么都不传)

|



内核态 红黑树(常驻) → 就绪链表

| fd 就绪时由回调

| 直接挂到就绪链表

返回就绪链表里那 5 个

$O(n) \rightarrow O(\text{就绪数})$ 。1 万连接里只有 5 个活跃时，差距是 2000 倍。

11.4 【面试】水平触发 LT vs 边缘触发 ET

水平触发 (Level Triggered, 默认)

「只要缓冲区里还有数据，每次 `epoll_wait` 都会通知你」

- ☒ 你这次没读完，下次还会提醒 → 不容易写错
- ☒ 没读完会反复触发，理论上效率略低

边缘触发 (Edge Triggered, `EPOLLET`)

「只在状态发生变化时通知一次」(从「没数据」变成「有数据」)

- ☒ 通知次数少，效率高
- ☒ 必须一次把数据读干净，否则剩下的数据永远不会再触发通知 → 连接假死

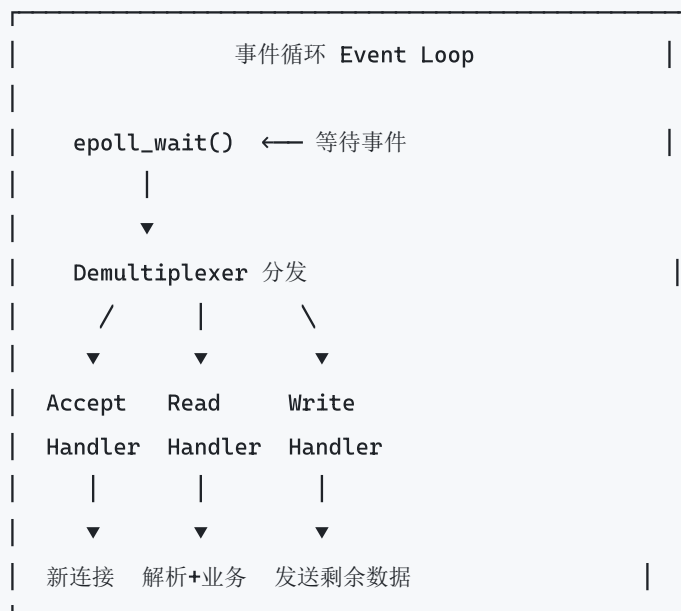
用 ET 的三条铁律

1. fd 必须是非阻塞的 (否则最后一次 `read` 会永久阻塞整个事件循环)
2. 必须循环读到 `EAGAIN` / `EWOULDBLOCK`
3. 写也一样：写到 `EAGAIN`，然后注册 `EPOLLOUT` 等下次可写

```
// ET 模式下的正确读法
for (;;) {
    ssize_t n = read(fd, buf, sizeof(buf));
    if (n > 0) { process(buf, n); continue; } // 继续读
    if (n == 0) { close_conn(); break; } // 对端关闭
    if (errno == EINTR) continue;
    if (errno == EAGAIN) break; // ★ 读干净了, 正常退出
    close_conn(); break; // 真错误
}
```

经验：先用 LT 把功能写对，确实成为瓶颈了再考虑 ET。Nginx 用 ET，Redis 用 LT —— 两个都是顶级项目。

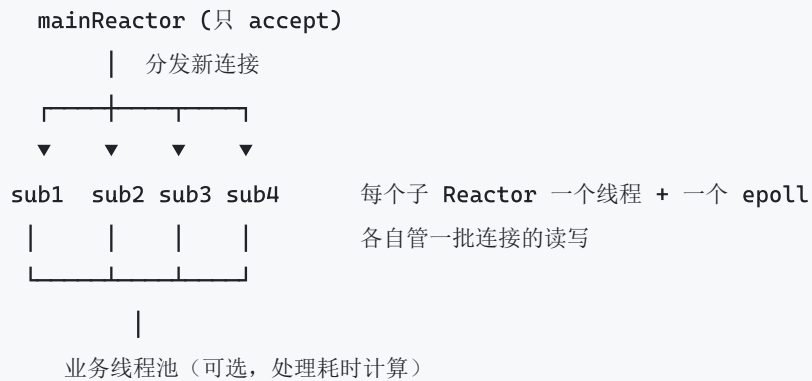
11.5 Reactor 模式



三种变体

模型	结构	代表
单 Reactor 单线程	一个线程做全部事情	Redis
单 Reactor 多线程	IO 在主线程，业务丢线程池	业务较重的服务
主从 Reactor 多线程	主 Reactor 只 accept，分给多个子 Reactor 各跑事件循环	Nginx / Netty / muduo

主从 Reactor:



Proactor vs Reactor

- **Reactor**: 「可以读了, 你自己去读」 (就绪通知)
- **Proactor**: 「我已经帮你读好了, 数据在这」 (完成通知)

Windows IOCP 和 Linux io_uring 属于 Proactor。

11.6 事件驱动编程的实际难点

① 代码从线性变状态机

```
// 阻塞式: 线性, 好读
auto req = read_request(fd);
auto resp = handle(req);
write_response(fd, resp);

// 事件式: 要把「读了一半」「处理中」「没发完」都存成显式状态
struct Conn {
    std::string inbuf;           // 收到但还没解析完的
    std::string outbuf;         // 想发但内核缓冲区满了的
    State state;
};
```

缓解手段: 协程 (C++20 coroutine / Go goroutine) —— 把线性写法还给你, 底层还是事件驱动。

② 绝对不能在事件循环里做阻塞操作

一次同步的数据库查询、一次文件读、一次 DNS 解析, 就会**卡住所有连接**。要么异步化, 要么丢给线程池。

③ 写缓冲区管理

```
// send 可能只发出去一部分
ssize_t n = send(fd, data, len, 0);
if (n < (ssize_t)len) {
    conn.outbuf.append(data + n, len - n);    // 剩下的存起来
    enable_write_event(fd);                  // 注册 EPOLLOUT
}
// 可写事件到来时继续发，发完了要【取消】EPOLLOUT
// ★ 忘了取消的话，LT 模式下会一直触发，CPU 直接跑满 100%
```

④ 惊群 (Thundering Herd)

多个进程/线程 epoll 同一个监听 fd，来一个连接**全被唤醒**，但只有一个能 accept 成功。

解决：EPOLLEXCLUSIVE (Linux 4.5+) 或 SO_REUSEPORT (每个进程一个独立监听 fd，内核做负载均衡)。

⑤ 定时器

事件循环里要处理超时（空闲连接踢下线、请求超时、重试）。

常见做法：- **小顶堆 / 时间轮** 存所有定时任务，把最近的超时时间作为 `epoll_wait` 的 timeout - Linux 还可以用 `timerfd` 把定时器变成一个 fd，一起 epoll

第 12 章 · 综合实战：迷你 HTTP 服务器

► 对应程序： `ch12_http_server`

```
build\bin\ch12_http_server.exe 8080
# 浏览器打开 http://127.0.0.1:8080
curl http://127.0.0.1:8080/api/time
curl -X POST -d "hello" http://127.0.0.1:8080/api/echo
curl -H "Authorization: Bearer secret123" http://127.0.0.1:8080/admin/panel
```

不到 400 行 C++，把前面所有东西串起来。

12.1 HTTP 协议格式

请求	响应
GET /path?query HTTP/1.1\r\n	HTTP/1.1 200 OK\r\n
Host: example.com\r\n	Content-Type: text/html\r\n
Content-Length: 5\r\n	Content-Length: 13\r\n
\r\n	\r\n
hello	Hello

HTTP 是怎么解决「粘包」的：

- 1. `\r\n\r\n` 作为分隔符，标记头部结束 —— 先找到它，才知道头部收全了
- 2. `Content-Length` 告诉你正文有多长 —— 按这个长度收满
- 3. 也可以用 `Transfer-Encoding: chunked`（分块传输，长度未知时用）

这正是第 9 章讲的「分隔符 + 长度前缀」混合方案。

12.2 解析器的关键：可重入

```
// 返回 nullptr = 数据还不完整，请继续收
// 有值 = 解析成功，并【从 buf 里消费掉】这个请求
std::optional<HttpRequest> try_parse(std::string& buf) {
    size_t header_end = buf.find("\r\n\r\n");
    if (header_end == std::string::npos) {
        if (buf.size() > 16 * 1024) throw std::runtime_error("请求头过大"); // ★ 防 DoS
        return std::nullopt; // 头部还没收全
    }

    // ... 解析请求行和头部 ...

    size_t content_length = /* 从头部取 */;
    if (content_length > 8 * 1024 * 1024) throw std::runtime_error("请求体过大"); // ★
    if (buf.size() < header_end + 4 + content_length) {
        return std::nullopt; // body 还没收全
    }

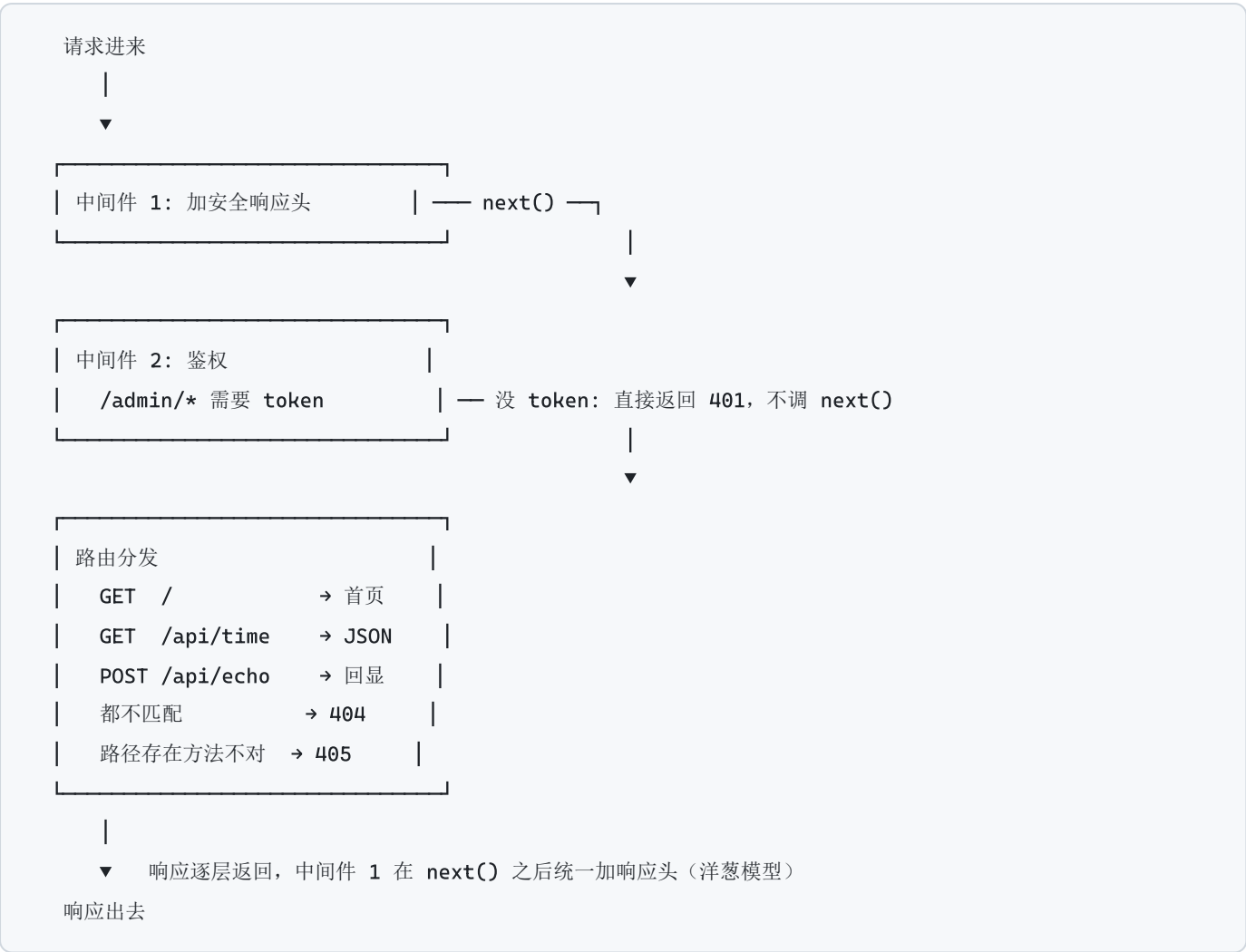
    req.body = buf.substr(header_end + 4, content_length);
    buf.erase(0, header_end + 4 + content_length); // ★ 消费掉
    return req;
}
```

调用方的循环

```
for (;;) {
    // 先尽量从已有缓冲里解析（一次 recv 可能收到多个请求 — HTTP 流水线）
    while (auto req = try_parse(buf)) {
        HttpResponse res;
        router.handle(*req, res);
        send_all(conn, res.serialize());
    }
    // 缓冲里没有完整请求了，再去读
    long n = recv(conn, chunk, sizeof(chunk), 0);
    if (n <= 0) break;
    buf.append(chunk, n);
}
```

★ 两处长度校验必不可少，否则恶意客户端发一个超长请求头或 Content-Length: 4294967295 就能把你的内存吃光。

12.3 架构：中间件 + 路由



代码里用到的模式：

模式	用在哪
责任链	中间件管道
策略	每个路由的 Handler 是一个 <code>std::function</code>
建造者	<code>HttpResponse</code> 的链式 <code>.set_status().json()</code>
RAII	<code>netc::Socket</code> 自动 close

用到的现代 C++：

```
std::optional<HttpRequest>           // 「可能解析不出来」
std::function<void(...)>               // Handler / Middleware
std::string_view                      // 零拷贝解析
auto [k, v] : headers                 // 结构化绑定
lambda + 移动捕获                    // 路由注册
std::shared_ptr<netc::Socket>         // 跨线程转移连接所有权
```

12.4 HTTP 关键概念

长连接 (keep-alive)

HTTP/1.1 默认长连接：一条 TCP 连接上可以发多个请求。这是**最重要的性能优化**——省掉每次的三次握手（1 RTT）和 TIME_WAIT。

短连接：[握手][请求1][响应1][挥手] [握手][请求2][响应2][挥手]
长连接：[握手][请求1][响应1][请求2][响应2][请求3][响应3][挥手]

服务端要在响应里带 `Connection: keep-alive`，客户端发 `Connection: close` 时才断开。

常用状态码

码	含义	什么时候用
200	OK	成功
201	Created	POST 创建成功
204	No Content	成功但无正文 (DELETE)
301/302	重定向	永久 / 临时
304	Not Modified	缓存有效
400	Bad Request	请求格式错误
401	Unauthorized	未认证 (还没登录)
403	Forbidden	已认证但无权限
404	Not Found	资源不存在
405	Method Not Allowed	路径对但方法不对
429	Too Many Requests	限流
500	Internal Server Error	服务端出错
502	Bad Gateway	网关拿到上游的坏响应
503	Service Unavailable	服务暂时不可用 (过载/维护)
504	Gateway Timeout	网关等上游超时

HTTP 版本演进

版本	关键改进	底层
HTTP/1.0	每请求一连接	TCP
HTTP/1.1	长连接、管线化、Host 头、分块传输	TCP
HTTP/2	二进制分帧 、多路复用（一条连接跑多个流）、头部压缩 HPACK、服务端推送	TCP
HTTP/3	基于 QUIC(UDP) 、0-RTT 握手、 彻底解决队头阻塞 、连接迁移	UDP

队头阻塞 (Head-of-Line Blocking)：HTTP/2 虽然在应用层做了多路复用，但底层 TCP 一旦丢包，**所有流都要等重传**。HTTP/3 用 UDP + QUIC，每个流独立，一个流丢包不影响其它流。

12.5 从这里往下走

这个迷你服务器**缺什么**（也就是你可以继续练的方向）：

- 1. **改成事件驱动**：用第 11 章的 `epoll` 替换线程池，扛住 1 万连接
- 2. **HTTPS**：接 OpenSSL，理解 TLS 握手
- 3. **静态文件服务**：`sendfile` 零拷贝、`Range` 断点续传、`ETag` 缓存
- 4. **Transfer-Encoding: chunked**：流式响应
- 5. **WebSocket**：HTTP 升级握手 + 帧协议
- 6. **连接超时与限流**：定时器轮、令牌桶
- 7. **压测**：`wrk -t4 -c100 -d30s http://127.0.0.1:8080/`，看瓶颈在哪

推荐研读的开源实现（按难度排序）：

项目	规模	看什么
<code>cpp-http-lib</code>	单头文件	简洁的 HTTP 实现
muduo	中等	陈硕的 Reactor 网络库，中文注释和配套书，最适合学习
<code>libuv</code>	中等	跨平台事件循环（Node.js 的底座）
Redis	中等	单线程事件循环的极致， <code>ae.c</code> 只有几百行
Nginx	大	工业级主从 Reactor + 模块化
Boost.Asio	大	现代 C++ 异步模型 + 协程支持

附录 A · 四周复习计划

按每天 2~3 小时估算。赶时间就跳过标 ☆ 的部分。

第 1 周：语法回炉

天	内容	动手任务
1	第 0 章全部	跑 <code>ch00_refresher</code> ，把「指针 vs 引用」那节的代码默写一遍
2	1.1~1.4 (auto / decltype / 移动语义 / 完美转发)	自己写一个带资源的类，实现五个特殊成员函数，用打印验证什么时候调哪个
3	1.5~1.7 (初始化 / lambda / nullptr 等)	用 lambda 重写几个 STL 算法调用
4	1.8~1.10 (constexpr / 范围 for / 智能指针)	把一段用 <code>new/delete</code> 的老代码改成智能指针
5	1.11~1.14 (变参模板 / default delete / 多线程)	写一个 10 行的线程安全计数器，对比有锁/无锁/无保护三种结果
6	第 2 章 C++14 全部	写一个用移动捕获的异步任务
7	复习 + 第 1 周小结	合上文档，默写：移动构造、完美转发模板、RAII 类

第 2 周：现代特性 + 标准库

天	内容	动手任务
8	3.1~3.5 (结构化绑定 / if 初始化 / CTAD / if constexpr / 折叠)	用 <code>if constexpr</code> 写一个通用的 <code>to_string</code>
9	3.6~3.8 (optional / variant / string_view)	用 <code>variant</code> 写一个表达式求值器
10	3.9~3.11 (filesystem / 并行算法 / 杂项)	写个小工具：递归统计目录下各类文件的数量和大小
11	4.1~4.2 (concepts / ranges)	用 <code>ranges</code> 重写一段 for 循环密集的数据处理
12	4.3~4.6 (协程 / <code><=></code> / span / format)	手写一个 Generator，产出素数序列
13	第 6 章 STL 上半 (容器)	写个小 benchmark：vector vs list vs deque 遍历/插入耗时
14	第 6 章 STL 下半 (算法 / 字符串 / chrono / random)	不看文档，用算法库完成：找出数组中出现次数最多的 3 个元素

第 3 周：设计模式 + 网络基础

天	内容	动手任务
15	7.1~7.3 (RAII / Pimpl / CRTP / 类型擦除)	给一个现有类加 Pimpl
16	7.4~7.5 (创建型 + 结构型)	实现一个注册式工厂，加新类型不改工厂
17	7.6~7.11 (行为型)	用 variant 实现一个订单状态机
18	8.1~8.5 (什么是网络 / 分层 / IP 端口 / 字节序)	跑 ch08_net_basics，用 hexdump 观察字节序
19	8.6~8.10 (sockaddr / DNS / TCP vs UDP / 握手挥手)	用 Wireshark 抓一次 curl example.com，找出三次握手的三个包
20	8.11~8.16 (状态机 / socket API / 选项 / 排障)	ss -tanp 观察本机各种连接状态；故意不设 SO_REUSEADDR 重现「地址已占用」
21	复习 + 第 3 周小结	手画：三次握手、四次挥手、TCP 状态机

第 4 周：网络编程实战

天	内容	动手任务
22	第 9 章 TCP 上半 (服务端套路 / 三种并发模型)	跑三种模式，用两个客户端验证 iterative 模式确实会阻塞
23	第 9 章 TCP 下半 (粘包 / 客户端 / 实测)	自己实现长度前缀协议，替换掉回显逻辑
24	第 10 章 UDP	用 UDP 写一个简易「局域网设备发现」(广播 + 应答)
25	11.1~11.3 (为什么要多路复用 / 三者对比 / 三种写法)	跑 select 和 poll 版聊天室，加一个 /who 命令列出在线用户
26	11.4~11.6 (LT vs ET / Reactor / 实际难点)	☆ 在 WSL 上跑 epoll 版；把 LT 改成 ET，故意只读一次，观察连接假死
27	第 12 章 HTTP 上半 (协议 / 解析器 / 架构)	加一个新路由 + 一个新中间件 (比如请求计数限流)
28	第 12 章 HTTP 下半 + 总复习	☆ 把线程池换成 epoll 事件循环；用 wrk 压测对比 QPS

附录 B · 练习题（由易到难）

B.1 语法与 STL

1. 写一个 `SafeVector<T>`，`operator[]` 带边界检查（越界抛异常），其余接口转发给内部 `std::vector`。要求支持移动、不支持拷贝。
2. 实现 `template <typename... Ts> std::string join(const char* sep, Ts&&... args)`，用折叠表达式把任意个可打印参数拼成字符串。
3. 用 `std::variant` 实现一个 JSON 值类型：`using Json = std::variant<std::nullptr_t, bool, double, std::string, std::vector<Json>, std::map<std::string, Json>>`；（提示：递归类型需要包一层 `struct`）写一个 `to_string(const Json&)`。
4. 不用 `std::sort`，自己实现一个满足 `std::sortable` 概念约束的快排。
5. 写一个 LRU 缓存：`get / put` 都是 $O(1)$ 。（提示：`std::list + std::unordered_map<K, list::iterator>`）

B.2 设计模式

6. 实现一个线程安全的事件总线：`bus.subscribe<OrderCreated>(handler) / bus.publish(OrderCreated{...})`。要求订阅者用 `weak_ptr` 持有，失效自动清理。
7. 用类型擦除实现一个 `AnyCallable`，能装下任意签名兼容的可调用对象，支持拷贝，且不用 `std::function`。
8. 给第 12 章的 HTTP 服务器加一个装饰器：给任意 Handler 套上「记录耗时」的外壳。

B.3 网络编程

9. 长度前缀协议：改造 `ch09_tcp`，实现 [4字节长度][JSON 消息体] 的协议。要求处理任意的拆包/粘包情况（可以在客户端故意一次只发 1 字节来测试）。
10. 心跳与超时：给聊天室服务器加上「30 秒没消息就踢下线」。用一个小顶堆管理超时时间，作为 `epoll_wait` 的 `timeout` 参数。
11. 文件传输：写一个能传大文件（>2GB）的客户端/服务端。要求显示进度、支持断点续传（协议里带 `offset`）、校验 MD5。
12. 简易 RPC：定义 [4字节长度][4字节请求ID][方法名\0][参数...]，服务端维护一个 `map<string, function>` 分发，客户端支持并发请求（靠请求 ID 匹配响应）。

13. ☆ **把 HTTP 服务器改成 epoll 事件驱动**。难点：写缓冲区管理（发不完的数据要存起来 + 注册 EPOLLOUT）。做完用 `wrk -t4 -c1000 -d30s` 对比改造前后的 QPS。
14. ☆ **实现最小可用的 WebSocket**：HTTP Upgrade 握手（算 Sec-WebSocket-Accept）+ 帧解析（掩码、分片、opcode）。
-

附录 C · 高频面试题清单

C++ 语言

- `auto` 的推导规则？`auto&&` 和 `T&&` 有什么关系？
- 移动语义解决了什么问题？为什么移动构造要加 `noexcept`？
- `std::move` 到底做了什么？（答：只是个 `static_cast`）
- 完美转发的原理？引用折叠规则？为什么必须用 `std::forward`？
- `unique_ptr` 和 `shared_ptr` 的区别？`shared_ptr` 线程安全吗？（答：控制块的引用计数是原子的，但指向的对象不是）
- 循环引用怎么产生、怎么解决？
- 虚函数怎么实现的？虚表在哪？为什么构造函数不能是虚函数？
- 为什么基类析构要 `virtual`？什么时候可以不 `virtual`？
- 什么是对象切片？
- `const` 成员函数能修改成员吗？（答：`mutable` 成员可以）
- 空类的 `sizeof` 是多少？（答：1，为了保证不同对象地址不同）
- 五法则 / 零法则是什么？
- `if constexpr` 和普通 `if` 的区别？
- `std::optional` / `variant` / `any` 各自的适用场景？
- 静态局部变量的初始化是线程安全的吗？（答：C++11 起是）

STL

- `vector` 扩容策略？为什么是 1.5 或 2 倍？扩容后迭代器还有效吗？
- `map` 和 `unordered_map` 怎么选？各自最坏复杂度？
- `emplace_back` 和 `push_back` 的区别？
- 什么时候迭代器会失效？（各容器分别说）
- `std::remove` 真的删除元素了吗？
- `accumulate` 和 `reduce` 的区别？

并发

- 五种内存序的区别？什么时候能用 `relaxed` ？
- 条件变量为什么要带谓词？什么是虚假唤醒？
- 死锁的四个必要条件？ 怎么避免？
- 什么是伪共享（false sharing）？ 怎么解决？（答：cache line 对齐）
- 无锁编程的 ABA 问题？

网络（重点）

- 三次握手为什么不是两次？ 不是四次？
 - 四次挥手为什么是四次？
 - **TIME_WAIT 的作用？ 为什么是 2MSL？ 大量 TIME_WAIT 怎么办？**
 - **大量 CLOSE_WAIT 说明什么？**（答：应用层收到 FIN 后没 close，是 bug）
 - TCP 怎么保证可靠传输？（序号 + 确认 + 重传 + 校验和 + 流控 + 拥塞控制）
 - 流量控制和拥塞控制的区别？（答：**流控是照顾接收方**，靠滑动窗口；**拥塞控制是照顾网络**，靠拥塞窗口）
 - 拥塞控制四个阶段？（慢启动、拥塞避免、快重传、快恢复）
 - **粘包是什么？ 三种解决方案？**
 - `select` / `poll` / `epoll` 的区别？ `epoll` 为什么快？
 - **LT 和 ET 的区别？ 用 ET 有什么注意事项？**
 - Reactor 和 Proactor 的区别？
 - 什么是 C10K 问题？
 - `recv` 返回 0 意味着什么？ 返回 -1 呢？
 - 为什么 `send` 要循环调用？
 - `SO_REUSEADDR` 和 `SO_REUSEPORT` 的区别？
 - Nagle 算法是什么？ 什么时候要关掉？
 - HTTP/1.1 vs HTTP/2 vs HTTP/3 的关键差异？
 - 什么是队头阻塞？ HTTP/2 和 HTTP/3 分别怎么处理？
 - 从浏览器输入 URL 到页面显示，发生了什么？（DNS → TCP 握手 → TLS 握手 → HTTP 请求 → 服务端处理 → 响应 → 解析 HTML → 加载资源 → 渲染）
-

附录 D · 推荐资源

书

书	适合阶段	说明
《Effective Modern C++》Scott Meyers	现在就看	42 条现代 C++ 最佳实践，本文档很多内容的出处
《C++ Primer》(第5版)	查漏补缺	大部头，当字典用
《UNIX 网络编程 卷1》Stevens	网络必读	socket 编程圣经，虽然老但概念不过时
《TCP/IP 详解 卷1》Stevens	深入协议	想搞懂协议细节看这本
《Linux 多线程服务端编程》陈硕	实战首选	中文，muduo 作者，讲清楚了为什么这么设计
《C++ Concurrency in Action》(第2版)	并发深入	内存模型讲得最清楚
《C++ Templates》(第2版)	模板深入	需要写库的时候看

网站

- cppreference.com —— 唯一权威的在线手册，有中文版但建议看英文
- isocpp.org/faq —— 官方 FAQ
- **C++ Core Guidelines** —— Bjarne 和 Herb Sutter 维护的编码规范
- **Compiler Explorer (godbolt.org)** —— 看代码编译成什么汇编，学优化必备
- quick-bench.com —— 在线微基准测试
- cppinsights.io —— 看编译器把 lambda / 范围 for / 模板展开成了什么，学语法糖神器

视频 / 会议

- **CppCon** (YouTube) —— 每年最重要的 C++ 会议
- Back to Basics 系列 —— CppCon 的入门专题，讲得非常清楚

开源项目（按学习难度排序）

项目	学什么
cpp-http-lib	单头文件 HTTP 库，简洁
muduo	Reactor 模型的教科书实现，配套中文书
libuv	跨平台事件循环（Node.js 底座）
Redis (ae.c)	极简事件循环，几百行
fmt	std::format 的原型，学现代 C++ 库设计
Nginx	工业级主从 Reactor
Boost.Asio	现代异步模型 + 协程

附录 E · 工程速查

编译命令

```
# MSVC
cl /std:c++20 /utf-8 /W4 /EHsc /Zc:__cplusplus /permissive- main.cpp

# GCC / Clang
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 main.cpp

# 开发期强烈建议加 sanitizer
g++ -std=c++20 -g -fsanitize=address,undefined main.cpp
```

CMake 最小模板

```
cmake_minimum_required(VERSION 3.20)
project(MyProject LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(app main.cpp)

if (MSVC)
    target_compile_options(app PRIVATE /utf-8 /W4 /Zc:__cplusplus /permissive-)
    target_link_libraries(app PRIVATE ws2_32)          # 网络程序需要
else()
    target_compile_options(app PRIVATE -Wall -Wextra)
    find_package(Threads REQUIRED)
    target_link_libraries(app PRIVATE Threads::Threads)
endif()
```

本工程的命令

```
scripts\build_vs.bat          :: 生成 .sln + 编译全部
build\bin\ch01_cpp11.exe      :: 运行某一章

:: 或用 VS 「打开文件夹」 直接识别 CMakeLists.txt
```

```
# WSL / Linux
cmake -B build-linux -DCMAKE_BUILD_TYPE=Debug
cmake --build build-linux -j$(nproc)
./build-linux/bin/ch11_multiplex 8890 epoll
```

文档到此结束。

记住一件事：**这份文档的价值不在于读完，而在于配套代码你改过多少遍。** 把 `src/` 下的程序当成沙盒，随便改、随便崩，改坏了 `git checkout` 就回来了。